

AUTONOMOUS MOBILE ROBOT NAVIGATION WITH DYNAMIC OBSTACLE AVOIDANCE IN INDOOR ENVIRONMENTS

Thesis submitted by

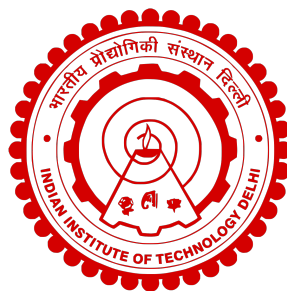
Pawan Kumar
2024JRB2031

under the guidance of

Prof. S.K. Saha
Indian Institute of Technology Delhi

*in partial fulfilment of the requirements
for the award of the degree of*

Master of Technology in Robotics



Center of Excellence on BIRD
(Biologically Inspired Robots and Drones)

INDIAN INSTITUTE OF TECHNOLOGY DELHI

May 2026

THESIS CERTIFICATE

This is to certify that the thesis titled **AUTONOMOUS MOBILE ROBOT NAVIGATION WITH DYNAMIC OBSTACLE AVOIDANCE IN INDOOR ENVIRONMENTS**, submitted by **Pawan Kumar (2024JRB2031)**, to the Indian Institute of Technology, Delhi, for the award of the degree of **Master of Technology**, is a bonafide record of the work done by him under our supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.



Prof. S.K. Saha

Department of Mechanical Engineering

Indian Institute of Technology Delhi

Date: **June 2026**

*Dedicated to my parents,
whose unwavering faith and sacrifice
made this journey possible,
and to every engineer who believes
that robots can make the world safer.*

ACKNOWLEDGEMENTS

I am deeply grateful to my supervisor, **Professor S.K. Saha**, for his invaluable guidance, patient mentorship, and constant encouragement throughout this project. His insightful direction — particularly the vision for large-scale multi-map navigation using map-tiling — transformed this work from a demonstration in a limited environment into a framework with genuine industrial potential. His emphasis on rigorous experimentation and honest reporting of limitations has shaped my approach to engineering research in a lasting way.

I sincerely thank to **Prof. S.K Saha** and the faculty and staff of the **Robotics** at IIT Delhi for providing access to the Novus AMR 500 robot platform. The **Lecture Hall Complex(LHC)** environment provided both the physical space and the technical ecosystem required to test the system under real-world indoor conditions.

My heartfelt appreciation goes to my fellow M.Tech Robotics batchmates, whose collaborative spirit and willingness to troubleshoot hardware failures and ROS configuration issues at any hour made the long laboratory sessions productive and enjoyable. Special thanks to those who helped carry and reset the Novus robot during the many field trials.

I am indebted to my parents for their unconditional support, sacrifice, and belief in my abilities. Their encouragement during challenging phases of this project provided the strength to persevere.

Finally, I acknowledge the open-source ROS community - without whose collective effort in developing `move_base`, AMCL, the TEB local planner, and countless supporting packages, a project of this scope would not have been feasible within the M.Tech timeframe.

Pawan Kumar

2024JRB2031

ABSTRACT

KEYWORDS: Autonomous Mobile Robot, ROS Navigation Stack, AMCL, TEB Local Planner, Dynamic Obstacle Avoidance, Multi-Sensor Fusion, Velodyne C16, Costmap, Indoor Navigation

This thesis presents the design, implementation, and experimental validation of an autonomous mobile robot navigation system with real-time dynamic obstacle avoidance, developed for the **Novus AMR 500** differential-drive robot operating in indoor environments. The work addresses the critical gap between laboratory-grade SLAM demonstrations and deployment-ready autonomous navigation in real, human-occupied spaces.

Building upon a foundational 2D SLAM-based system from M.Tech Project-I, this project introduces a comprehensive upgrade to a multi-sensor 3D perception pipeline. The robot is equipped with a **Velodyne C16 360° 3D LiDAR** and an **Intel RealSense D457 depth camera**, providing complementary obstacle detection over height ranges from 0.1 m to 2.2 m. Sensor data is fused through a unified costmap obstacle layer, enabling detection of humans, furniture, and overhead obstacles that a single 2D laser scan would miss entirely.

Adaptive Monte Carlo Localization (AMCL) replaces the previous SLAM during navigation approach, eliminating a 9-second initialization delay and achieving sub-0.2-second localization on pre-built 2D occupancy grid maps. The **Timed Elastic Band (TEB) local planner** is adopted over the standard Dynamic Window Approach (DWA), generating kinematically feasible, time-optimal trajectories that dynamically deform around moving obstacles at 20 Hz.

A custom **velocity smoother** node ensures gradual velocity ramping (0.05 m/s^2 linear, 0.3 rad/s^2 angular), preventing wheel slippage and odometry drift inherent in large differential-drive platforms. Navigation parameters - including AMCL particle counts, costmap update frequencies, inflation radius, and cost scaling - were carefully tuned to enable reliable navigation on large-scale maps (8000×8000 pixels) through corridors as narrow as 2 m.

A key forward-looking contribution is the design and prototyping of a **topological multi-map navigation system**. The system uses a **map_manager** node to partition large environments into overlapping map tiles and switches the active AMCL map as the robot

crosses tile boundaries, using polygon-based transition zones and a **rigid-body coordinate transform** to re-localize the robot across independently mapped regions.

Experimental results on a real indoor testbed demonstrate successful autonomous navigation with dynamic obstacle avoidance, reliable AMCL localization, and smooth motion control. The system provides a modular, configurable, and deployment-ready navigation stack for large-scale indoor mobile robot applications.

Contents

ACKNOWLEDGEMENTS	i
ABSTRACT	ii
LIST OF TABLES	ix
LIST OF FIGURES	x
ABBREVIATIONS	xi
NOTATION	xii
1 INTRODUCTION	1
2 RELATED WORK	4
2.1 Mobile Robot Localization	4
2.2 Simultaneous Localization and Mapping	4
2.3 Path Planning	5
2.4 Dynamic Obstacle Avoidance	6
2.5 Multi-Map Navigation	6
2.6 ROS Navigation Stack	7
3 SYSTEM ARCHITECTURE AND HARDWARE	8
3.1 Overview of the Novus AMR 500	8
3.2 Safety Instructions and Operational Warnings	9
3.3 LED Status Indicators	9
3.4 Hardware Components	9
3.4.1 Sensor Suite	9
3.4.2 Front and Rear Views	11
3.4.3 Safety Components	11

3.5	Basic Operations	13
3.5.1	Operating Procedure and Pre-checks	13
3.5.2	Power-On Procedure	14
3.5.3	Power-Off Procedure	14
3.5.4	Modes of Operation	14
3.6	Software Integration and ROS Configuration	15
3.6.1	Network Configuration	15
3.6.2	Master-Slave ROS Configuration	15
3.6.3	Accessing Sensor Data and Actuators	16
3.6.4	ROS Node Architecture	17
3.7	Robot Specifications	18
4	MAPPING AND LOCALIZATION	20
4.1	Occupancy Grid Mapping	20
4.2	2D SLAM with GMapping (Project-I)	20
4.3	AMCL Localization	21
4.3.1	Particle Filter Overview	21
4.3.2	AMCL Configuration	22
4.3.3	2D Laser Scan from 3D LiDAR	22
4.3.4	Localization Performance	22
5	NAVIGATION SYSTEM AND MULTI-MAP ARCHITECTURE	23
5.1	Navigation Stack Overview	23
5.2	Global Planner: NavFn (Dijkstra's Algorithm)	23
5.3	Local Planner: Timed Elastic Band (TEB)	24
5.3.1	TEB Configuration for Novus AMR 500	24
5.4	Velocity Smoother	25
5.5	Multi-Map Navigation Framework	25
5.5.1	Motivation: Limitation of Single-Map Navigation	25
5.5.2	Environment Mapping with GMapping	26
5.5.3	The Coordinate Frame Mismatch Problem	26

5.5.4	Proposed Solution: Landmark-Based Rigid-Body Coordinate Transformation	27
5.5.5	Transition Zone Design: Trigger and Drop Polygons	29
5.5.6	Rigid-Body Coordinate Transform Derivation	30
5.5.7	Map Manager State Machine	30
5.5.8	Map Region Bounds	31
5.6	Autonomous Navigation Results	32
6	DYNAMIC OBSTACLE AVOIDANCE AND 3D PERCEPTION	33
6.1	Costmap Architecture	33
6.2	Sensor Fusion for Obstacle Detection	33
6.2.1	Height Filtering	33
6.2.2	Obstacle Layer Fusion	34
6.3	Inflation Layer	34
6.4	Dynamic Obstacle Clearing: The Ghost Obstacle Problem	35
6.4.1	Problem Description	35
6.4.2	Root Cause Analysis	35
6.4.3	Solution: LaserScan Clearing Source + Auto-Clearer	35
6.5	Costmap Configuration Summary	36
6.6	Sensor Data Synchronization	36
7	MTP-1 EXPERIMENTAL RESULTS (PROJECT-I BASELINE)	37
7.1	Project-I System Configuration	37
7.2	Test Environment	37
7.3	Test-MTP1-01: GMapping Initialization and SLAM Startup	38
7.4	Test-MTP1-02: 20 m Straight-Path Navigation Accuracy	38
7.5	Test-MTP1-03: Static Obstacle Avoidance on 20 m Path	39
7.6	Test-MTP1-04: E-Stop Safety Response	41
7.7	Project-I Summary and Limitations Identified	41
8	EXPERIMENTAL RESULTS AND DISCUSSION	43
8.1	Test Environment	43
8.2	Localization Performance	43

8.2.1	Startup Time	43
8.2.2	AMCL Convergence	44
8.3	Navigation Performance	44
8.3.1	Single-Map Navigation	44
8.3.2	Dynamic Obstacle Avoidance	44
8.3.3	Smooth Motion Control	45
8.4	Multi-Map Navigation	45
8.4.1	Map Switch Procedure Timing	45
8.4.2	Relocalization Accuracy	45
8.5	Comparison: Project-I vs Project-II	46
9	ISO 3691-4 PERFORMANCE VALIDATION	47
9.1	Test Configuration	47
9.1.1	Test Environment	47
9.1.2	Hardware and Software Configuration	47
9.2	Test-01: Same-Map Navigation Accuracy	47
9.3	Test-02: Cross-Map Navigation (Map Switching)	48
9.4	Test-03: Re-Localization Accuracy After Map Switch	49
9.5	Test-04: Static Obstacle Avoidance	49
9.6	Test-05: Dynamic Obstacle Avoidance	50
9.7	Test-06: Path Following and Trajectory Accuracy (ATE)	50
9.8	Test-07: Emergency Stop and Safety Response	51
9.9	Test-08: End-to-End Full Mission	51
9.10	Overall ISO Compliance Summary	52
9.10.1	Key Observations	52
9.10.2	Known Limitations	53
10	CONCLUSION AND FUTURE WORK	54
10.1	Summary of Work	54
10.2	Limitations	55
10.3	Future Work	55
10.4	Closing Remarks	56

List of Tables

3.1	Safety symbols and their descriptions as defined in the Novus User Guide.	9
3.2	LED indicator states and their operational meanings.	10
3.3	LSLiDAR (Velodyne C16) specifications.	11
3.4	Intel RealSense D457 depth camera specifications.	11
3.5	Key ROS topics published by the Novus AMR 500.	17
3.6	Complete hardware specifications of the Novus AMR 500.	19
4.1	Key AMCL Configuration Parameters	22
5.1	Key TEB local planner parameters for the Novus AMR 500.	24
5.2	Trigger and drop polygon vertices for both map transition directions. Each row pair shares the same physical corners measured in different map coordinate frames.	29
5.3	Map region coordinate bounds used by the map manager to classify goal poses.	32
6.1	Height Filtering Parameters for Obstacle Detection	34
6.2	Obstacle Layer Sensor Configuration	34
6.3	Costmap Configuration Summary	36
7.1	Project-I (MTP-1) Navigation System Configuration	37
7.2	Test-MTP1-01: GMapping SLAM Initialization Performance	38
7.3	Test-MTP1-02: 20 m Navigation Accuracy - Position Error (Project-I) . . .	39
7.4	Test-MTP1-02: 20 m Navigation Accuracy - Yaw Error (Project-I)	39
7.5	Test-MTP1-03: Static Obstacle Avoidance - 20 m Path in SAC Hall (Project-I)	40
7.6	Test-MTP1-04: Emergency Stop Response (Project-I)	41
7.7	Project-I ISO-Style Compliance Summary	41
8.1	Localization Performance Comparison: Project-I vs Project-II	43
8.2	Multi-Map Switch Procedure Timing	45

8.3	Comprehensive Comparison of Project-I and Project-II Navigation Systems	46
9.1	Hardware and Software Test Configuration	48
9.2	Test-01: Same-Map Navigation Accuracy Results	48
9.3	Test-02: Cross-Map Navigation Summary	48
9.4	Test-03: Re-Localization Accuracy After Map Switch	49
9.5	Test-04: Static Obstacle Avoidance Results	50
9.6	Test-05: Dynamic Obstacle Avoidance Results	50
9.7	Test-06: Path Following and Trajectory Accuracy	51
9.8	Test-07: Emergency Stop Response	51
9.9	Test-08: End-to-End Full Mission Results	52
9.10	ISO 3691-4 Compliance Summary - All 8 Tests Passed	52

List of Figures

3.1	The Novus AMR 500 autonomous mobile robot platform used in this thesis.	8
3.2	Preinstalled sensors on the Novus AMR 500.	10
3.3	Front view of the Novus AMR 500 showing the LSLiDAR, RealSense camera, and front bumper.	12
3.4	Rear view of the Novus AMR 500 showing the HMI panel, mode selector, E-stop, and battery compartment.	12
3.5	Field of view of the safety laser scanners and scanning height diagram showing warning zone and stop zone boundaries.	13
3.6	Master-Slave ROS configuration: the <code>/etc/hosts</code> file on the development PC and the required <code>ROS_MASTER_URI</code> and <code>ROS_IP</code> environment variables. . . .	16
5.1	Occupancy grid map of the experimental environment generated by GMapping SLAM. White cells are free space, black cells are obstacles (walls), and grey cells are unexplored regions. The map spans the operational corridors and rooms used for all navigation experiments.	26
5.2	Both Map01 and Map02 overlaid in RViz. The maps appear misaligned (rotated and translated relative to each other) because they were built in separate GMapping sessions with different robot starting poses, resulting in inconsistent coordinate frame origins and orientations.	27
5.3	Proposed multi-map navigation architecture. Two maps are built with an overlapping common region. Four corresponding corner points in the overlap define the trigger polygon (Map01 coordinates) and drop polygon (Map02 coordinates). When the robot enters the trigger polygon, a 2D rigid-body transform is computed from these four point-pairs to re-localize the robot instantly in the target map's coordinate frame.	28
5.4	RViz screenshot during an autonomous navigation experiment on the Novus AMR 500. The global plan (green path) is computed by NavFn; the local costmap (coloured overlay) captures dynamic obstacles; the robot pose (red arrow) is maintained by AMCL. The navigation system successfully guides the robot along the planned path while avoiding obstacles in real time. . .	32

ABBREVIATIONS

AMR	Autonomous Mobile Robot
ROS	Robot Operating System
AMCL	Adaptive Monte Carlo Localization
SLAM	Simultaneous Localization and Mapping
TEB	Timed Elastic Band
DWA	Dynamic Window Approach
MCL	Monte Carlo Localization
LiDAR	Light Detection and Ranging
IMU	Inertial Measurement Unit
ICP	Iterative Closest Point
TF	Transform (ROS coordinate frame system)
NavFn	Navigation Function (Dijkstra-based global planner)
OGM	Occupancy Grid Map
2D	Two-Dimensional
3D	Three-Dimensional
FOV	Field of View
URDF	Unified Robot Description Format
CoE	Centre of Excellence
BIRD	Building Innovative Research and Development
CPU	Central Processing Unit

NOTATION

(x, y, θ)	Robot pose: position (x, y) and heading θ , m and rad
v	Linear (forward) velocity, m/s
ω	Angular (yaw) velocity, rad/s
a	Linear acceleration, m/s ²
α	Angular acceleration, rad/s ²
r	Wheel radius, m
L	Wheelbase (distance between drive wheels), m
N	Encoder ticks per wheel revolution
$p(\cdot)$	Probability density function
$\mathbf{x}_t$	Robot state at time t
$\mathbf{z}_t$	Sensor measurement at time t
$\mathbf{u}_t$	Control input (velocity command) at time t
$\mathbf{m}$	Map
w_i	Particle weight in MCL filter
K_v	Linear proportional gain
K_ω	Angular proportional gain
r_{inf}	Costmap inflation radius, m
(tx, ty, θ_t)	Rigid-body transform: translation and rotation angle
d_H	Hausdorff distance, m
ATE	Absolute Trajectory Error, m

Chapter 1

INTRODUCTION

Autonomous navigation is a foundational capability for mobile robots deployed in real-world indoor environments. A robot that can navigate independently - localizing itself on a map, planning collision-free paths, and reacting to dynamic obstacles such as moving humans - provides substantial value in applications including hospital logistics, warehouse automation, office delivery, and facility monitoring. Despite decades of research in mobile robotics, the transition from controlled laboratory demonstrations to reliable deployment in populated, large-scale buildings remains a significant engineering challenge.

Indoor environments present a unique combination of difficulties. Unlike outdoor settings, GPS-based localization is unavailable, requiring the robot to maintain its position estimate entirely from onboard sensors. The geometry of indoor spaces - long corridors, symmetric rooms, glass partitions - can confuse laser-based localization. Human foot traffic introduces dynamic, unpredictable obstacles that change the environment continuously. Large facilities, such as hospital wings or university buildings, span areas far exceeding the practical limits of a single SLAM-generated map, imposing computational and memory constraints on the navigation stack.

This thesis documents the design, implementation, and experimental validation of a comprehensive autonomous navigation system for the **Novus AMR 500**, a large differential-drive mobile robot platform ($1.17\text{ m} \times 0.75\text{ m} \times 0.425\text{ m}$). The work directly continues and extends **M.Tech Project-I**, which established a basic 2D SLAM-based navigation capability using a single 2D laser scan. Project-II fundamentally upgrades every layer of the navigation stack - from perception through planning to motion control - and introduces a novel multi-map navigation architecture designed for large, multi-room indoor environments.

Motivation

The limitations observed at the end of Project-I defined the research agenda for Project-II:

- **9-second initialization delay:** The use of SLAM (GMapping) as the localization algorithm required a full scan-matching warm-up before the robot could begin moving, making rapid deployment impractical.
- **Single 2D laser plane:** The 2D laser scan captured only a horizontal slice of the environment at approximately 0.3 m height. Obstacles above or below this plane -

human legs, overhanging signs, table-top objects - were invisible to the navigation system.

- **Jerky motion and wheel slippage:** The DWA local planner issued abrupt velocity step commands directly to the motor controllers. The Novus robot's weight (95 kg payload capacity) and wheel friction characteristics caused slippage during sudden acceleration, corrupting odometry and causing localization drift.
- **Ghost obstacle accumulation:** Dynamic obstacles (people walking through the room) left persistent traces on the costmap after they had moved away, causing the robot to plan unnecessarily long detours or become stuck.
- **Single-map scalability:** A single occupancy grid map for a large building (8000×8000 pixels at 0.05 m/cell) imposed excessive CPU and memory load on the onboard computer, and AMCL particle updates became unreliable on such large grids.

Contributions

This thesis makes the following technical contributions:

1. **AMCL-based instant localization:** Replacing SLAM-during-navigation with Adaptive Monte Carlo Localization (AMCL) on a pre-built map eliminates the initialization delay, achieving sub-0.2second localization startup.
2. **Multi-sensor 3D obstacle perception:** A dual-sensor architecture combining the Velodyne C16 360° 3D LiDAR (primary, long-range) and Intel RealSense D457 depth camera (secondary, close-range front-facing) provides comprehensive obstacle detection across height ranges 0.1-2.2 m, covering humans, furniture, and overhead obstacles.
3. **TEB local planner for dynamic avoidance:** The Timed Elastic Band (TEB) local planner replaces DWA, generating kinematically feasible, time-optimal trajectories that deform around dynamic obstacles in real time at 20 Hz.
4. **Custom velocity smoother:** A ROS node that intercepts and ramps raw velocity commands from the planner (0.05 m/s^2 linear, 0.3 rad/s^2 angular) prevents wheel slippage, reducing odometry corruption and localization drift.
5. **Costmap architecture optimization:** A carefully tuned two-costmap (global static + local dynamic) architecture, with sensor fusion through a unified obstacle layer, enables navigation on large-scale maps through 2 m-wide corridors.
6. **Multi-map navigation framework:** A topological map-manager node that monitors robot position, triggers map switches when transition polygons are entered, and re-localizes the robot using a closed-form 2D rigid-body coordinate transform computed from polygon correspondences between separately mapped regions.

Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 reviews the relevant literature in mobile robot localization, path planning, obstacle avoidance, and multi-map navigation. Chapter 3 describes the hardware platform, sensor suite, and software architecture. Chapter 4 presents the mapping and localization subsystem, including AMCL configuration and the SLAM pipeline from Project-I. Chapter 5 details the navigation system, including the planning hierarchy, multi-map architecture, and coordinate transform derivation. Chapter 6 covers the 3D obstacle perception, costmap configuration, and dynamic obstacle clearing. Chapter 8 presents experimental results and quantitative performance evaluation. Chapter 10 concludes with a summary of contributions and directions for future work.

Chapter 2

RELATED WORK

This chapter reviews the state of the art in the areas most directly relevant to this thesis: mobile robot localization, path planning, dynamic obstacle avoidance, multi-map navigation, and the ROS navigation stack. The foundational treatment of robot kinematics, dynamics, and coordinate transformations follows [1].

2.1 Mobile Robot Localization

Localization - the problem of estimating a robot's pose within a known map - is the foundation of autonomous navigation. Early approaches relied on deterministic methods such as dead reckoning, which accumulates errors over time, and landmark-based triangulation, which requires sparse, identifiable features in the environment [2].

Monte Carlo Localization (MCL) introduced a particle filter framework for mobile robot localization [3, 4]. A set of weighted particles, each representing a possible robot pose, is propagated through a motion model and reweighted by a sensor measurement model. MCL handles multimodal distributions and recovers from localization failures, making it well-suited for indoor environments with symmetries. **Adaptive Monte Carlo Localization (AMCL)** [5] extends MCL by dynamically adjusting the number of particles using the KLD-sampling technique [6], reducing computation when the distribution is concentrated and expanding it during uncertainty. AMCL is the de facto standard localization algorithm in the ROS navigation stack [7] and is the primary localization method used in this thesis.

The **likelihood field** sensor model [2] is used by AMCL to score laser scan measurements against a precomputed occupancy grid map. It computes the probability of observed scan endpoints by looking up their distance to the nearest occupied cell in the map, providing efficient and robust scan matching without requiring explicit ray-casting.

2.2 Simultaneous Localization and Mapping

When no pre-built map is available, SLAM algorithms construct the map and localize simultaneously. **GMapping** [8], used in this project's predecessor (Project-I), applies

Rao-Blackwellized particle filters where each particle carries a full map hypothesis. It is effective for 2D laser-based indoor mapping but requires a 2D laser scan as input and accumulates error during long traversals without loop closure.

Hector SLAM [9] performs high-frequency scan matching via Gauss-Newton optimization on a multi-resolution occupancy grid, requiring no odometry. It is suitable for platforms where wheel odometry is unreliable. Indian researchers have successfully applied and evaluated these GMapping and Hector SLAM algorithms for autonomous mobile robots deployed in domestic Indian environments [10], confirming their viability for indoor navigation tasks on hardware configurations representative of the Indian robotics ecosystem.

RTAB-Map [11] is a general-purpose visual and LiDAR SLAM system with memory management and loop closure detection. It supports 3D occupancy grid generation directly from point clouds, making it appropriate for environments where 3D obstacle representation is needed. The 3D mapping capability of RTAB-Map was evaluated in this project as a potential upgrade (Project-II background work) before the decision was made to retain 2D maps for AMCL and use 3D sensor data only for costmap obstacle marking.

2.3 Path Planning

Path planning in mobile robotics is typically decomposed into **global planning** (computing an optimal route on the static map) and **local planning** (real-time trajectory generation around dynamic obstacles).

Dijkstra’s algorithm and its heuristic variant **A*** [12] are the most widely used global planners. The ROS **NavFn** package implements Dijkstra’s algorithm on the 2D costmap, computing a cost-minimizing path from start to goal. **D*-Lite** [13] extends A* for dynamic replanning in changing environments.

For local planning, the **Dynamic Window Approach (DWA)** [14] samples feasible velocity commands within a dynamic window constrained by the robot’s acceleration limits, selects commands that maximize a scoring function balancing goal progress, clearance, and velocity. DWA is computationally efficient but can struggle with narrow passages and complex dynamic obstacle interactions.

The **Timed Elastic Band (TEB)** algorithm [15, 16] represents the trajectory as a sequence of robot poses with associated time intervals (a “rubber band” stretched between start and goal) and optimizes it using sparse nonlinear programming. TEB explicitly handles kinematic constraints (non-holonomic motion), obstacle clearance, and velocity/acceleration limits, generating smooth, time-optimal trajectories. Its support for dynamic obstacles and homotopy-class planning (exploring multiple topologically distinct path classes) makes it

particularly effective in environments with moving people. TEB is adopted as the local planner in this thesis.

2.4 Dynamic Obstacle Avoidance

Dynamic obstacle avoidance is the ability to detect, track, and navigate around obstacles that move during the robot’s mission. Early reactive methods such as **Potential Fields** [17] apply virtual repulsive forces from obstacles, but suffer from local minima. **Velocity Obstacles (VO)** [18] and its extension **Reciprocal Velocity Obstacles (RVO)** [19] model the set of velocities that would lead to collision and select a velocity outside this set, enabling smooth multi-agent avoidance.

In the ROS costmap framework [7, 20], sensor measurements are projected onto a 2D obstacle layer. A comprehensive survey of indoor AMR navigation challenges - including dynamic obstacle handling in real Indian warehouse and campus environments - is presented by Trivedi and Bhatt [21], confirming that ghost-obstacle accumulation and single-plane laser limitations are among the most practically significant issues in deployed systems. Research from Indian institutions further demonstrates that neuro-fuzzy controllers can effectively guide autonomous robots around both static and dynamic obstacles in real-world conditions [22], providing complementary algorithmic context to the reactive planner approach adopted in this thesis. The **inflation layer** expands obstacle cells by a configurable radius, creating a cost gradient that planners use to maintain safe clearance. The **voxel layer** extends this to 3D by maintaining a volumetric occupancy grid and projecting occupied voxels down onto the 2D costmap [23]. This thesis uses both 2D obstacle layer and voxel layer configurations, ultimately selecting the obstacle layer for production deployment due to more reliable clearing behavior.

2.5 Multi-Map Navigation

Large-scale indoor navigation across multiple rooms or floors requires partitioning the environment into manageable sub-maps. **Topological maps** represent the environment as a graph of locations and connections, enabling efficient high-level planning but lacking metric accuracy for local navigation [24]. **Metric map tiling** divides a large occupancy grid into spatially adjacent tiles, each managed independently by a separate localization instance [25].

The challenge in multi-map systems is **seamless transition**: when the robot crosses a tile boundary, it must re-localize on the new map immediately without losing position accuracy. This requires knowledge of the geometric relationship between the coordinate

frames of adjacent maps. When maps are built in separate SLAM sessions, the coordinate origins differ and a **rigid-body transformation** must be computed to align them [26].

This thesis proposes a novel approach using **transition polygon correspondences**: the same physical region (a doorway or corridor segment) is identified in both maps as a quadrilateral polygon, and the rigid-body transform (translation + rotation) is computed analytically from the centroid and orientation of the two polygon sets. This approach requires no prior knowledge of the global map layout beyond the polygon vertex coordinates.

2.6 ROS Navigation Stack

The **Robot Operating System (ROS)** navigation stack [7, 27] provides an integrated framework for autonomous robot navigation, comprising costmap management, global and local path planning, and recovery behaviors. The practical deployment of ROS-based AMRs in Indian warehouse environments is documented by Nair et al. [28], demonstrating that the ROS navigation stack is well-suited to Indian industrial settings with appropriate sensor and planner configuration. Coverage path planning - a closely related application - has been studied by Veerajagadheswar et al. [29] in the context of Indian research infrastructure, providing additional validation of LiDAR-based autonomous navigation in structured indoor environments. The `move_base` action server serves as the central orchestrator, accepting goal poses and driving the robot via velocity commands. Its modular plugin architecture allows the planner and costmap layers to be selected and configured at launch time, making it the standard deployment platform for ROS-based mobile robots.

The TF (transform) library [30] maintains the real-time tree of coordinate frame transformations required by all ROS navigation components, including the critical `map`→`odom`→`base_link` chain. AMCL publishes the `map`→`odom` transform, and the base controller publishes `odom`→`base_link`. Ensuring this chain remains valid during map switches is a key challenge addressed in this thesis.

The experimental robot platform used throughout this work is the **Novus AMR 500** manufactured by The Hi-Tech Robotic Systemz Ltd., Gurugram, India [31]. This platform provides infrastructure-free natural SLAM-based navigation, a 500 kg payload capacity, 360-degree safety sensing, and a WiFi-connected fleet management interface, making it representative of the state of the art in Indian industrial AMR deployments.

Chapter 3

SYSTEM ARCHITECTURE AND HARDWARE

This chapter presents a thorough description of the experimental platform used throughout this thesis: the **Novus AMR 500** autonomous mobile robot. It covers the robot's physical construction, sensor suite, safety systems, operational procedures, ROS-based software integration, and complete hardware specifications. A solid understanding of this platform is essential context for the algorithmic contributions described in subsequent chapters.

3.1 Overview of the Novus AMR 500

The Novus AMR 500 is an industrial-grade Autonomous Mobile Robot (AMR) developed by **Novus Hi-Tech Robotic Systemz Ltd.** It is designed for indoor logistics, material transport, and research applications in structured environments. The platform employs a differential-drive kinematic model and is built to carry payloads of up to 500 kg, making it suitable for heavy-duty warehouse and facility automation.

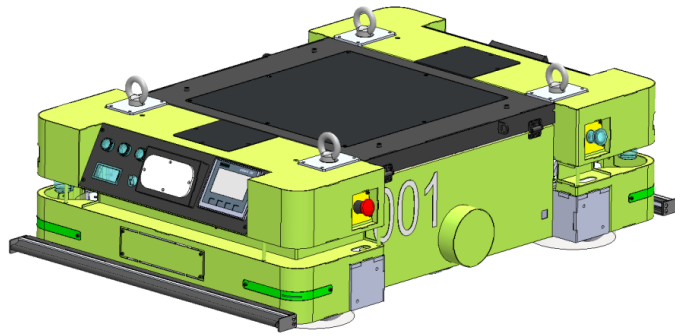


Figure 3.1: The Novus AMR 500 autonomous mobile robot platform used in this thesis.

As shown in Figure 3.1, the Novus robot features a rugged rectangular chassis with four corner-mounted LED indicator strips, a Human Machine Interface (HMI) touchscreen, and an integrated sensor payload at the front. The onboard compute unit is an Intel NUC running Ubuntu 20.04 with ROS Noetic, providing the full ROS navigation stack directly on the robot. The robot communicates with an external development PC over a wired Ethernet LAN, using a ROS Master-Slave architecture described in Section 3.6.

3.2 Safety Instructions and Operational Warnings

Before operating the Novus robot, all users must be fully familiar with the safety classification defined in the user guide. Table 3.1 summarises the three-level warning hierarchy used throughout the manual and in field operations.

Symbol	Description
CAUTION	Indicates a situation which can lead to minor or moderate personal injury if precautionary measures are ignored.
WARNING	Indicates a situation which can lead to severe personal injury if precautionary measures are ignored.
DANGER	Indicates a situation which can lead to death or severe personal injury if precautionary measures are ignored.

Table 3.1: Safety symbols and their descriptions as defined in the Novus User Guide.

Key pre-operation safety requirements:

- The operator must visually inspect the robot before each session to verify all components are securely mounted.
- Battery voltage must exceed 25 V (verified with a multimeter) before connection.
- The robot track must be clear of all obstructions; no loose cables should be present.
- Emergency stop buttons must be accessible and functional at all times.
- The robot must be brought to a complete halt before powering off.

3.3 LED Status Indicators

The Novus robot features LED strips at its four corners that provide real-time operational status. Table 3.2 details the meaning of each LED state. These indicators are particularly important during autonomous navigation experiments, as they provide the first visual indication of safety zone breaches or error conditions.

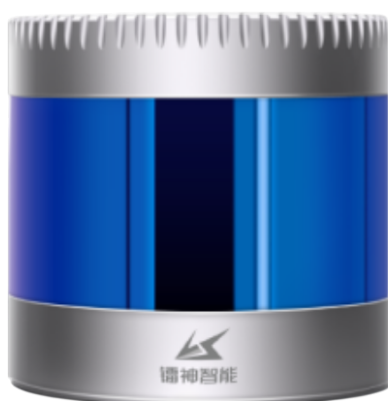
3.4 Hardware Components

3.4.1 Sensor Suite

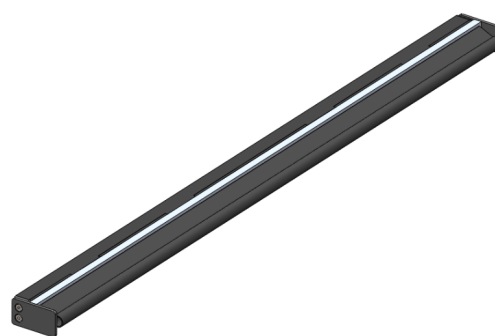
The Novus robot is equipped with two complementary preinstalled sensors that together provide comprehensive spatial perception of the environment (Figure 3.2).

LED State	Meaning
RED SOLID	Error in robot operation.
RED BLINK	Low battery.
GREEN SOLID	Robot is in AUTO mode and ready for autonomous navigation.
GREEN BLINK	Robot is in Manual mode, or in Auto mode waiting for the Start button.
YELLOW SOLID	Robot stop zone has been breached; robot has halted.
YELLOW BLINK	Robot's warning zone has been breached.

Table 3.2: LED indicator states and their operational meanings.



(a) LSLiDAR (3D LiDAR)



(b) Intel RealSense Depth Camera

Figure 3.2: Preinstalled sensors on the Novus AMR 500.

LSLiDAR - 3D LiDAR (Primary Sensor)

The **LSLiDAR** (LS-series, equivalent to Velodyne VLP-16/C16) is a 16-channel rotating 3D LiDAR that provides full 360° horizontal field of view. It generates dense point-cloud data at 10 Hz, yielding approximately 300,000 points per second. In the ROS navigation stack, its output on `/velodyne_points` is used in two ways: (1) projected to a 2D laser scan via `pointcloud_to_laserscan` for AMCL localization, and (2) fed directly into the costmap 3D obstacle layer for volumetric obstacle detection. Table 3.3 lists its key parameters.

Intel RealSense D457 - Depth Camera (Secondary Sensor)

The **Intel RealSense D457** is a time-of-flight stereo depth camera mounted at the front of the robot. It provides dense depth measurements and RGB imagery in the 0.1-9 m frontal zone, complementing the LiDAR with higher point density at close range. This is critical for

Parameter	Value
Channels (vertical beams)	16
Horizontal FOV	360°
Vertical FOV	-15° to +15°
Angular Resolution (H)	0.1°-0.4°
Maximum Range	100 m
Update Rate	10 Hz
ROS Topic	/velodyne_points (PointCloud2)

Table 3.3: LSLiDAR (Velodyne C16) specifications.

detecting low-profile obstacles (table legs, suitcases, cables) that fall between the LiDAR's vertical beam angles. Table 3.4 summarises its specifications.

Parameter	Value
Sensor Technology	Active IR Stereo + RGB
Horizontal FOV	~ 87°
Minimum Range	0.1 m
Maximum Range	9 m
ROS Depth Topic	/camera_front/depth/color/points
ROS RGB Topic	/camera_front/color/image_raw

Table 3.4: Intel RealSense D457 depth camera specifications.

3.4.2 Front and Rear Views

Figures 3.3 and 3.4 show the front and rear physical layouts of the Novus robot, identifying the locations of sensors, controls, and safety components.

3.4.3 Safety Components

For enhanced operational safety, the robot is equipped with two types of protective sensors working in tandem.

Safety Laser Scanners

Two diagonally-mounted safety laser scanners (TIM - Time-of-flight Infrared Module) are installed at opposite corners of the robot chassis. These scanners continuously monitor a configurable protective field around the robot. When the stop zone is breached, the robot immediately halts all motion and activates the YELLOW SOLID LED state.

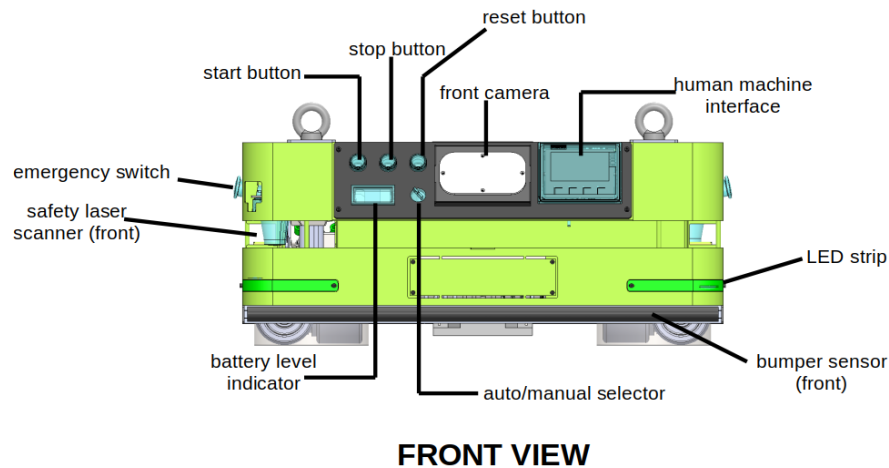


Figure 3.3: Front view of the Novus AMR 500 showing the LSLiDAR, RealSense camera, and front bumper.

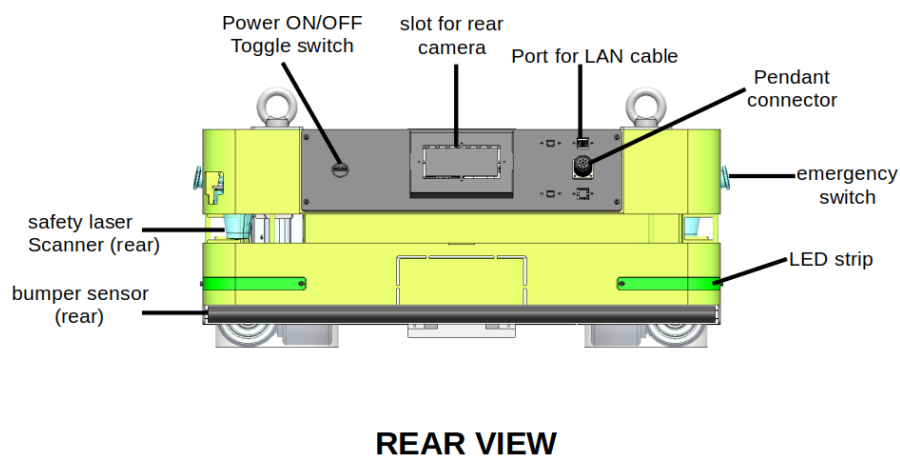


Figure 3.4: Rear view of the Novus AMR 500 showing the HMI panel, mode selector, E-stop, and battery compartment.

The scanners operate at the scan-plane height and detect obstacles that enter the defined warning or stop zones. They are only active in **Auto mode**; in Manual mode, these sensors are disabled and the operator is solely responsible for collision avoidance.

Bumper Sensors

Physical bumper sensors are installed at both the front and rear of the robot. These contact-based sensors serve as a fail-safe in case of safety laser scanner malfunction and are specifically designed to detect obstacles *lower* than the laser scan height - such as low-profile objects on the floor. Upon contact, the bumpers trigger an immediate stop.

Figure 3.5 illustrates the field of view of the dual safety laser scanners and their scanning height. The two scanners together achieve near-complete 360° coverage of the robot's perimeter. The predefined Field 1 configuration (selectable via HMI) is the recommended

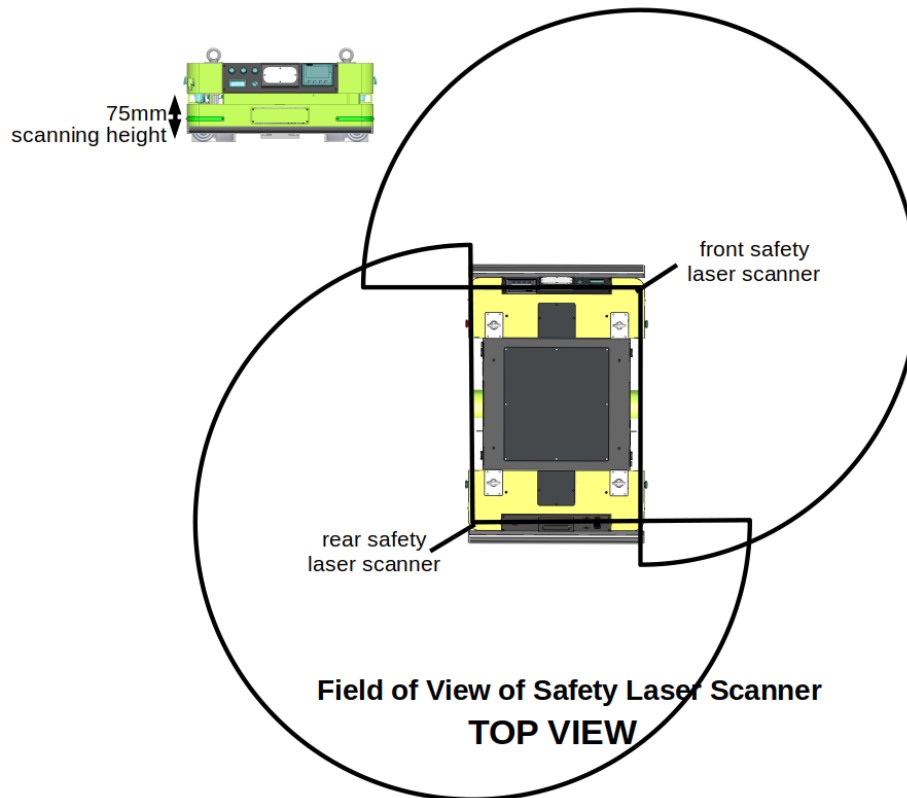


Figure 3.5: Field of view of the safety laser scanners and scanning height diagram showing warning zone and stop zone boundaries.

safety zone for all autonomous navigation experiments.

3.5 Basic Operations

3.5.1 Operating Procedure and Pre-checks

The following visual and electrical checks must be performed before *every* session:

1. Perform a visual inspection - verify all components are rigidly mounted.
2. Verify battery voltage exceeds 25 V using a multimeter.
3. Confirm the robot track is free from obstructions.
4. Confirm no loose cables are present on or around the robot.
5. Ensure battery connectors are attached properly and covers are closed.
6. Confirm the robot is in Manual Mode (toggle switch at rear).
7. Confirm all Emergency Stop switches are released.

3.5.2 Power-On Procedure

1. Toggle the Power ON/OFF switch to ON. The LED strips will display a boot pattern, the HMI will show a boot screen, and the battery level indicator will illuminate. If none of these occur within 10 seconds, check battery connections.
2. After boot completes (a few seconds), the blue push button at the rear will begin blinking. Press and hold this button for 2 seconds, twice, to arm the motor controller.
3. The LED strip turns **GREEN SOLID** when the robot is ready for AUTO mode operation.

Spare Error: If the robot displays an error on startup, it indicates the mode selector is in AUTO position during boot. Rotate the selector to Manual to resolve.

3.5.3 Power-Off Procedure

1. Ensure the robot has come to a complete halt before initiating shutdown.
2. Toggle the Power ON/OFF switch to the OFF state.

3.5.4 Modes of Operation

The Novus robot supports two operational modes, selectable via the rear-panel mode selector knob:

Manual Mode

Manual mode enables off-path maneuvering via the handheld Pendant controller. Safety laser scanners are **inactive** in this mode. Steps:

1. Power on the robot and set mode selector to Manual.
2. Press the Reset Push Button; wait 8 seconds for motor controller boot.
3. Use the Pendant buttons: **UP** (forward), **DW** (backward), **L/R** (turning).

In Manual mode, the robot expects velocity commands on `/cmd_vel_manual`.

Auto Mode

Auto mode enables fully autonomous, sensor-guided navigation. Safety laser scanners are **active**. Steps:

1. Configure the Remote PC (see Section 3.6).
2. Power on and set mode selector to Auto.

3. Press Reset; wait 8 seconds.
4. Publish an initial velocity command to `/cmd_vel_stage` (start with small values in m/s):

```
$ rostopic pub /cmd_vel_stage geometry_msgs/Twist \  
  "linear: {x: 0.0, y: 0.0, z: 0.0} angular: {x: 0.0, y: 0.0, z: 0.1}"
```

5. Press the Start Push Button.
6. Use the Emergency Stop button to halt the robot in any emergency.

Important: If the safety zone is breached during Auto mode, the robot halts and locks. To resume: switch to Manual → back to Auto → press Start.

3.6 Software Integration and ROS Configuration

The Novus robot runs **ROS Noetic** on Ubuntu 20.04 on its onboard Intel NUC. An external development PC (laptop or workstation) connects to the robot via a wired Ethernet LAN and acts as the ROS Slave node, while the robot's NUC acts as the ROS Master.

3.6.1 Network Configuration

Before connecting to the robot, the development PC must be configured to an IP address in the `192.168.100.xxx` subnet (excluding `192.168.100.104`, which is reserved for the robot):

1. Go to *Settings* → *Network* → click + to add a new profile.
2. Under *Identity*, assign a name to the connection.
3. Under *IPv4*, select **Manual** and set:
 - Address: `192.168.100.xxx` (choose any unused address)
 - Netmask: `255.255.255.0`
4. Click **Add** to save.

3.6.2 Master-Slave ROS Configuration

This step establishes the ROS communication channel between the development PC (Slave) and the Novus robot (Master). It is the critical prerequisite for all autonomous navigation experiments in this thesis.

1. Edit the `/etc/hosts` file on the development PC to add the robot's hostname:

```
$ cd /etc
$ sudo nano hosts
```

Add the following lines:

```
<IP_of_your_PC>    localhost
<IP_of_your_PC>    <YourComputerName>
192.168.100.104    nuc
```

2. Set the ROS environment variables in every new terminal session used for navigation:

```
$ export ROS_MASTER_URI=http://192.168.100.104:11311/
$ export ROS_IP=192.168.100.xxx    # IP of the development PC
```

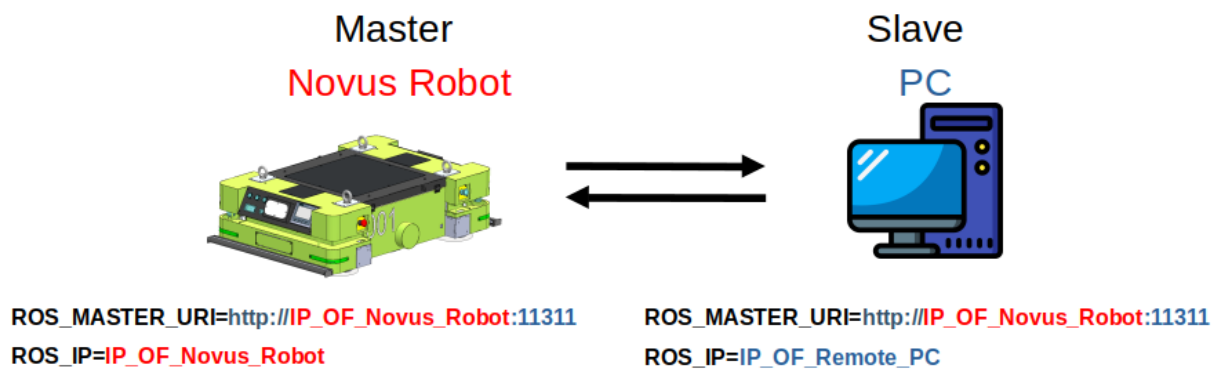


Figure 3.6: Master-Slave ROS configuration: the `/etc/hosts` file on the development PC and the required `ROS_MASTER_URI` and `ROS_IP` environment variables.

Figure 3.6 illustrates the configured hosts file and environment variable setup. The `ROS_MASTER_URI` directs every ROS node on the PC to the robot's ROS Master at `192.168.100.104:11311`. The `ROS_IP` variable ensures that the slave node advertises its correct network address so the Master can route messages back.

Key parameters:

- Robot IP address: `192.168.100.104`
- Robot hostname (NUC): `nuc`
- Robot ROS Master port: `11311`

3.6.3 Accessing Sensor Data and Actuators

Once the Master-Slave configuration is active, all topics published by the Novus robot are visible on the development PC. Run `rostopic list` to enumerate all available topics. Table 3.5 documents the key topics relevant to this thesis.

Subscribing to sensor data:

ROS Topic	Description
/velodyne_points	3D LiDAR point cloud (PointCloud2)
/camera_front/color/image_raw	RGB camera image
/camera_front/color/camera_info	RGB camera calibration
/camera_front/depth/camera_info	Depth camera calibration
/camera_front/depth/color/points	RGBD point cloud (PointCloud2)
/odom	Wheel encoder odometry
/tf / /tf_static	Transform tree
/cmd_vel_manual	Velocity command (Manual mode)
/cmd_vel_stage	Velocity command (Auto mode)
/robot_io_states	Digital I/O states (buttons, E-stop)

Table 3.5: Key ROS topics published by the Novus AMR 500.

```
# LiDAR 3D point cloud
$ rostopic echo /velodyne_points

# Front RGB camera
$ rostopic echo /camera_front/color/image_raw

# Robot I/O states (E-stop, start/stop buttons)
$ rostopic echo /robot_io_states
```

Publishing velocity commands:

```
# Manual mode
$ rostopic pub /cmd_vel_manual geometry_msgs/Twist \
  "linear: {x: 0.2, y: 0.0, z: 0.0} angular: {x: 0.0, y: 0.0, z: 0.0}"

# Auto mode
$ rostopic pub /cmd_vel_stage geometry_msgs/Twist \
  "linear: {x: 0.2, y: 0.0, z: 0.0} angular: {x: 0.0, y: 0.0, z: 0.0}"
```

3.6.4 ROS Node Architecture

The full navigation stack deployed for this thesis on top of the Novus platform consists of the following key ROS nodes:

- **velodyne_driver**: Receives raw UDP packets from the LSLiDAR and publishes /velodyne_points.

- `pointcloud_to_laserscan`: Projects the 3D cloud to a 2D `LaserScan` on `/velodyne_2d_scan` for AMCL.
- `amcl`: Particle-filter localization; publishes the `map→odom` TF transform and `/amcl_pose`.
- `map_server`: Serves the pre-built occupancy grid on `/map`.
- `move_base`: Central navigation server with NavFn global planner, TEB local planner, and dual costmaps.
- `nav_velocity_smoother`: Intercepts `/cmd_vel_raw`, applies acceleration ramping, and forwards smooth commands to `/cmd_vel_stage`.
- `auto_costmap_clearer`: Periodically clears costmaps to prevent ghost-obstacle accumulation.
- `map_manager`: Monitors AMCL pose, detects transition polygon entry, and manages autonomous map switching with rigid-body re-localization.

The TF tree hierarchy is:

$$\text{map} \xrightarrow{\text{AMCL}} \text{odom} \xrightarrow{\text{base ctrl}} \text{base_link} \xrightarrow{\text{URDF}} \text{laser, camera_front_link}$$

3.7 Robot Specifications

Table 3.6 presents the complete hardware specifications of the Novus AMR 500 as sourced from the official user guide.

The robot’s LiFePO4 battery chemistry was chosen for its thermal stability, long cycle life (1500+ cycles), and flat discharge curve - properties that ensure consistent power delivery throughout an 8-hour operational shift. The 24 V, 80 Ah capacity provides approximately 1.92 kWh of stored energy, sufficient for sustained indoor navigation tasks.

The 500 kg payload capacity, combined with a 250 kg unloaded weight, makes this one of the heaviest differential-drive AMR platforms available for academic research. This mass directly influenced the design decisions for the velocity smoother (Chapter 5), as sudden velocity commands cause significant wheel slip and odometry corruption on heavy platforms.

Category	Parameter	Value
Dimensions	Length	1100 mm
	Width	750 mm
	Height	410 mm
	Ground Clearance	35 mm
	Weight (unloaded)	250 kg
	Load Surface	600 mm × 830 mm
	Drive Wheel Dia.	210 mm
	Caster Wheel Dia.	100 mm
Payload	Maximum Payload	500 kg
Performance	Battery Run Time	8 hours
	Max Forward Speed	1.25 m/s
	Max Reverse Speed	1.25 m/s
	Acceleration	0.2 m/s ² (full payload, flat)
	Deceleration	0.2 m/s ² (full payload, flat)
Power	Battery Type	LiFePO4, 24 V, 80 Ah
	Charging Time	Up to 4 hours (0-80%: 3 hrs)
	External Charger	200-240 V AC, 50-60 Hz, Output: 24 V
	Battery Life Cycles	Minimum 1500 cycles
Compute	Onboard Computer	Intel NUC (x86-64)
	OS	Ubuntu 20.04 LTS
	Middleware	ROS Noetic

Table 3.6: Complete hardware specifications of the Novus AMR 500.

Chapter 4

MAPPING AND LOCALIZATION

Accurate localization - knowing *where* the robot is - is the prerequisite for all autonomous navigation. This chapter covers the map-building pipeline from M.Tech Project-I and the AMCL localization system central to Project-II.

4.1 Occupancy Grid Mapping

The navigation system uses a **2D occupancy grid map** as its environment representation. An occupancy grid divides the environment into a regular array of cells. Each cell stores a probability $p \in [0, 1]$ indicating whether it is:

- **Occupied** ($p > 0.65$): contains an obstacle (wall, furniture, door frame).
- **Free** ($p < 0.35$): traversable empty space.
- **Unknown** ($0.35 \leq p \leq 0.65$): not yet observed by the robot.

Maps are stored in two files: a **.pgm** image (grayscale pixel values encode occupancy) and a **.yaml** metadata file specifying the resolution, origin, and threshold values. The Novus robot's maps use a resolution of 0.05 m/cell (5 cm per pixel).

4.2 2D SLAM with GMapping (Project-I)

In M.Tech Project-I, **GMapping** [8] was used to build the occupancy grid map while simultaneously localizing the robot - the canonical SLAM problem. GMapping employs Rao-Blackwellized particle filters: each particle carries a full map hypothesis and a pose estimate. The algorithm proceeds as follows:

1. **Prediction:** Robot motion (from wheel odometry) is applied to each particle's pose estimate via the motion model, incorporating noise parameters that model encoder uncertainty.
2. **Scan matching:** Each particle's proposed pose is refined by locally optimizing scan alignment with its individual map hypothesis, reducing particle degeneracy.
3. **Weight update:** Particles are weighted by the likelihood of the observed laser scan given the particle's map and pose.

4. **Resampling:** Low-weight particles are eliminated and high-weight particles are replicated, concentrating probability mass on accurate hypotheses.

The 3D Velodyne point cloud was converted to a 2D laser scan using `pointcloud_to_laserscan` (horizontal slice at 0.3 m height) as input to GMapping. The resulting maps covered the test corridors at IIT Delhi with sufficient accuracy for AMCL-based navigation.

Limitation identified: Using GMapping as the runtime localization algorithm introduces a 9-second warm-up period before the robot can navigate. This delay is caused by the particle filter requiring several scan cycles to converge from the initial uniform distribution to a peaked pose estimate. This limitation motivated the transition to AMCL in Project-II.

4.3 AMCL Localization

Adaptive Monte Carlo Localization (AMCL) [5] is a particle filter-based localization algorithm that operates on a pre-built occupancy grid map. Unlike GMapping, AMCL does not update the map - it solely estimates the robot's pose within the fixed map. This separation of mapping and localization enables immediate startup with no initialization delay.

4.3.1 Particle Filter Overview

AMCL maintains a set of M particles $\mathcal{X}_t = \{(\mathbf{x}_t^{[i]}, w_t^{[i]})\}_{i=1}^M$, where $\mathbf{x}_t^{[i]}$ is a pose hypothesis and $w_t^{[i]}$ is its normalized weight. At each time step:

1. **Prediction:** Propagate each particle through the motion model:

$$\mathbf{x}_t^{[i]} \sim p(\mathbf{x}_t | \mathbf{u}_t, \mathbf{x}_{t-1}^{[i]})$$

where $\mathbf{u}_t$ is the odometry increment. The differential-drive motion model adds noise proportional to the translation and rotation to simulate odometry uncertainty.

2. **Weight update:** Score each particle against the laser scan:

$$w_t^{[i]} = p(\mathbf{z}_t | \mathbf{x}_t^{[i]}, \mathbf{m})$$

using the **likelihood field** sensor model, which computes the probability of each scan beam's endpoint given the occupancy grid map.

3. **Resampling:** Draw M' particles from $\mathcal{X}_t$ with replacement, weighted by $w_t^{[i]}$. The **KLD-sampling** criterion [6] adaptively sets M' : when particles are concentrated (high confidence), M' decreases; when particles are spread (high uncertainty), M' increases.

4.3.2 AMCL Configuration

The AMCL node was tuned for the Novus AMR 500 operating in IIT Delhi's indoor corridors. Table 4.1 lists the key parameters and their rationale.

Parameter	Value	Rationale
<code>min_particles</code>	100	Minimum particle count for efficiency
<code>max_particles</code>	500	Maximum count (reduced from 5000 to lower CPU ~60%)
<code>laser_max_beams</code>	100	Subsample 360° scan; balances accuracy vs. speed
<code>laser_model_type</code>	<code>likelihood_field</code>	Robust to minor map imperfections
<code>laser_sigma_hit</code>	0.2 m	Measurement noise standard deviation
<code>update_min_d</code>	0.1 m	Update filter every 10 cm translation
<code>update_min_a</code>	0.1 rad	Update filter every ~6° rotation
<code>odom_model_type</code>	<code>diff</code>	Differential-drive noise model
<code>odom_alpha1-4</code>	0.2	Wheel slip and encoder noise parameters
<code>transform_tolerance</code>	0.5 s	TF lookahead to prevent extrapolation errors

Table 4.1: Key AMCL Configuration Parameters

The reduction from 5000 to 500 maximum particles achieved approximately 60% CPU reduction with negligible localization accuracy loss in the structured indoor environment (straight corridors and rectangular rooms provide strong scan-matching constraints).

4.3.3 2D Laser Scan from 3D LiDAR

Since AMCL requires a 2D `sensor_msgs/LaserScan` input but the Velodyne C16 publishes 3D `PointCloud2`, the `pointcloud_to_laserscan` node converts the point cloud to a 2D scan by projecting all points within a height range of 0.3-1.0 m onto the horizontal plane and computing range values at each azimuth angle. This produces a dense 360° 2D scan at 10 Hz, providing richer scan data than a dedicated 2D LiDAR for AMCL matching.

4.3.4 Localization Performance

With the AMCL configuration described above, the robot achieves:

- **Startup time:** < 0.2 s (versus 9 s with GMapping in Project-I)
- **Steady-state position accuracy:** < 0.1 m in straight corridor segments
- **Angular accuracy:** < 5 heading error during normal operation

Localization failures (kidnapping, symmetric environment confusion) are handled by the AMCL recovery alpha parameters (`recovery_alpha_slow` = 0.001, `recovery_alpha_fast` = 0.1), which inject random particles when the average weight drops below a threshold.

Chapter 5

NAVIGATION SYSTEM AND MULTI-MAP ARCHITECTURE

This chapter details the complete navigation system: the two-layer path planning hierarchy, the custom velocity smoother, and - most significantly - the novel multi-map navigation framework designed for large-scale indoor environments that cannot be covered by a single SLAM-generated occupancy grid.

5.1 Navigation Stack Overview

The ROS `move_base` action server orchestrates autonomous navigation. Upon receiving a goal pose, it:

1. Calls the **global planner** (NavFn) to compute an optimal path on the static costmap.
2. Feeds the global path to the **local planner** (TEB), which generates kinematically feasible velocity commands at 20 Hz.
3. Monitors goal progress and invokes recovery behaviors if the robot becomes stuck.
4. Publishes `geometry_msgs/Twist` velocity commands on `/cmd_vel_raw`.

5.2 Global Planner: NavFn (Dijkstra's Algorithm)

The global planner **NavFn** applies Dijkstra's algorithm on the 2D global costmap to compute the minimum-cost path from the robot's current position to the goal. The costmap encodes traversability: free cells have low cost, inflated obstacle cells have high cost, and occupied cells are impassable.

Key configuration:

- `allow_unknown: true` - the robot is permitted to plan through unexplored (unknown) regions on the map, enabling navigation toward goals in unvisited areas.
- `default_tolerance: 0.25m` - if the exact goal cell is occupied, the planner accepts any cell within 0.25m.

5.3 Local Planner: Timed Elastic Band (TEB)

The **Timed Elastic Band** planner [16] represents the robot’s trajectory as a sequence of N intermediate poses $\{(x_k, y_k, \theta_k)\}_{k=1}^N$ interspersed with time differences $\{\Delta t_k\}$. The “elastic band” is stretched between the current pose (fixed) and the goal (fixed), and the intermediate poses are optimized to minimize a scalar cost:

$$J = \sum_k [w_t \cdot \Delta t_k + w_o \cdot c_{\text{obs}}(\mathbf{x}_k) + w_k \cdot c_{\text{kin}}(\mathbf{x}_k) + w_d \cdot c_{\text{dyn}}(\mathbf{x}_k)] \quad (5.1)$$

where c_{obs} penalizes proximity to static obstacles, c_{kin} enforces kinematic feasibility (forward-only driving, curvature limits), and c_{dyn} penalizes proximity to dynamic obstacles. The optimization is solved efficiently using the g2o sparse graph optimization framework.

5.3.1 TEB Configuration for Novus AMR 500

Table 5.1 lists the key TEB parameters configured for the Novus robot operating in indoor corridors.

Parameter	Value	Rationale
max_vel_x	0.12 m/s	Conservative for large robot mass
max_vel_x_backwards	0.01 m/s	Near-zero (avoids TEB singularity at 0)
max_vel_theta	0.10 rad/s	Slow rotation (AMCL loses track above 0.15)
acc_lim_x	0.05 m/s ²	Prevents wheel slip on smooth floors
acc_lim_theta	0.3 rad/s ²	Smooth angular acceleration
min_obstacle_dist	0.5 m	Safe clearance in narrow corridors
inflation_dist	0.8 m	Wide safety buffer
weight_kinematics_forward_drive	1000	Force forward-only driving
weight_obstacle	100	Strong obstacle avoidance
weight_dynamic_obstacle	100	Equal weight for dynamic obstacles
xy_goal_tolerance	0.25 m	Relaxed to reduce final oscillation
enable_homotopy_class_planning	true	Explore multiple path topologies

Table 5.1: Key TEB local planner parameters for the Novus AMR 500.

The TEB planner issues commands at 20 Hz directly to `/cmd_vel_raw`, which are then smoothed by the velocity smoother before reaching the robot base.

5.4 Velocity Smoother

A fundamental limitation of the standard ROS navigation stack is that velocity commands are issued as step changes. For a robot as massive as the Novus AMR 500 (250 kg unloaded), abrupt velocity steps cause wheel slippage, corrupting odometry and degrading AMCL localization accuracy.

The **velocity smoother node** (`nav_velocity_smoother.py`) intercepts commands on `/cmd_vel_raw` and publishes ramped commands on `/cmd_vel_stage`. Its ramp logic is:

$$v_{\text{cmd}}(t) = \text{clamp}(v_{\text{target}}, v_{\text{curr}} - a_{\text{lin}} \cdot \Delta t, v_{\text{curr}} + a_{\text{lin}} \cdot \Delta t) \quad (5.2)$$

$$\omega_{\text{cmd}}(t) = \text{clamp}(\omega_{\text{target}}, \omega_{\text{curr}} - a_{\text{ang}} \cdot \Delta t, \omega_{\text{curr}} + a_{\text{ang}} \cdot \Delta t) \quad (5.3)$$

where $a_{\text{lin}} = 0.05 \text{ m/s}^2$ and $a_{\text{ang}} = 0.3 \text{ rad/s}^2$ are the maximum permitted accelerations. Additional safety features:

- **Safety timeout:** If no new command is received for 0.5s, the smoother commands zero velocity.
- **Deadband:** Commands with $|v| < 0.005 \text{ m/s}$ are snapped to zero.
- **Update rate:** 30 Hz, providing finer temporal resolution than the TEB planner's 20 Hz output.

The full command pipeline is:

$$\text{move_base} \xrightarrow{\text{/cmd_vel_raw}} \text{velocity_smoother} \xrightarrow{\text{/cmd_vel_stage}} \text{robot base}$$

5.5 Multi-Map Navigation Framework

5.5.1 Motivation: Limitation of Single-Map Navigation

A 2D occupancy grid map covering a large building at 0.05 m/cell resolution may have dimensions of 8000×8000 pixels ($400 \text{ m} \times 400 \text{ m}$). AMCL must evaluate scan likelihoods on this grid for every particle at every update step. Even with only 500 particles, the memory footprint ($\sim 64 \text{ MB}$ for the map alone) and CPU cost grow prohibitively. Furthermore, GMapping cannot close loops over such distances without significant trajectory drift.

5.5.2 Environment Mapping with GMapping

The environment used in this thesis is the ground floor of the LHC(Lecture Hall Complex) building at IIT Delhi. The map was constructed using the GMapping SLAM algorithm, which builds a 2D occupancy grid from 2D laser scan data while simultaneously estimating the robot's trajectory using a Rao-Blackwellized particle filter.

Figure 5.1 shows the complete map generated by GMapping for the experimental environment.

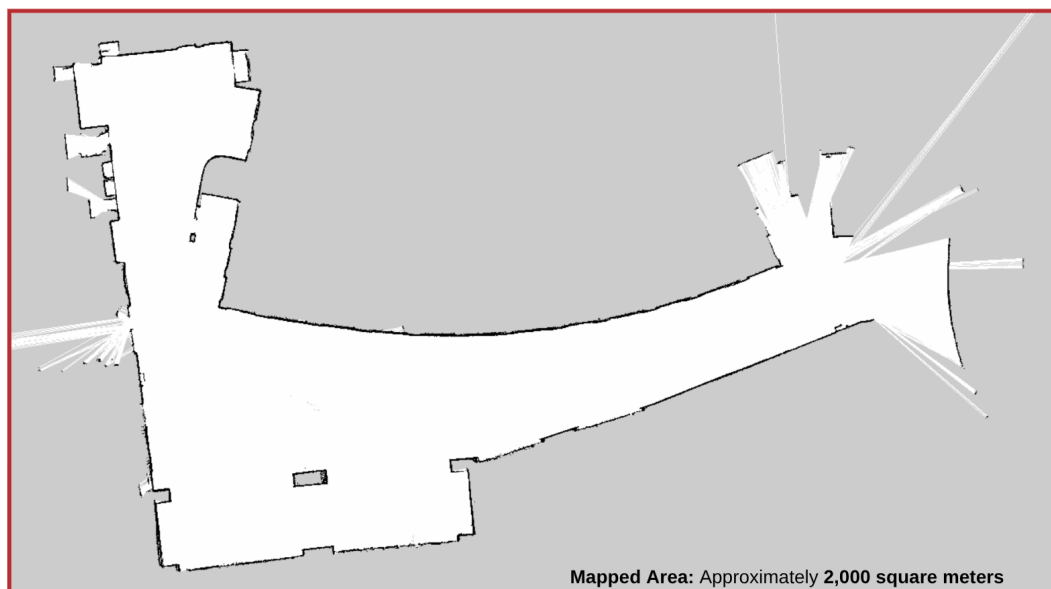


Figure 5.1: Occupancy grid map of the experimental environment generated by GMapping SLAM. White cells are free space, black cells are obstacles (walls), and grey cells are unexplored regions. The map spans the operational corridors and rooms used for all navigation experiments.

Due to the spatial extent of the environment, the map was partitioned into two separate tiles - **Map01** and **Map02** - each built in an independent GMapping session covering a distinct region of the building. This tiling approach, proposed by Prof. S.K. Saha, significantly reduces AMCL particle-filter computational cost and SLAM drift, while enabling navigation across the full building extent.

5.5.3 The Coordinate Frame Mismatch Problem

Figure 5.2 shows both map tiles visualised simultaneously in RViz. A striking feature is immediately apparent: the two maps are visibly *rotated and offset* relative to each other, even though they share a physical overlap region in the real building.

This misalignment is not a calibration error - it is an inherent consequence of the GMapping SLAM process. Each GMapping session initialises the robot's pose as the world



Figure 5.2: Both Map01 and Map02 overlaid in RViz. The maps appear misaligned (rotated and translated relative to each other) because they were built in separate GMapping sessions with different robot starting poses, resulting in inconsistent coordinate frame origins and orientations.

origin with the robot’s current heading defined as the $+x$ axis. Because the two sessions were started at different physical locations with different robot orientations, the map-frame coordinate system of Map01 and the map-frame coordinate system of Map02 are defined with different origins and rotations. The same physical corridor wall may have coordinates $(38.5, -15.2)$ in Map01’s frame and $(41.3, -16.1)$ in Map02’s frame, with a non-trivial rotation between the two frames.

A naive map switch - simply killing Map01’s `map_server` and launching Map02’s, then republishing the same AMCL pose - would place the robot at the wrong location in Map02’s coordinate system, causing immediate and total localization failure.

5.5.4 Proposed Solution: Landmark-Based Rigid-Body Coordinate Transformation

The solution developed in this thesis exploits the **physical overlap region** between the two maps. Since a portion of the real environment was mapped twice (once per session), there exist corresponding points in both maps that represent the same physical locations.

By identifying four such corresponding corners in the overlapping region and recording their coordinates in both map frames, a precise 2D rigid-body coordinate transform can be computed analytically.

Figure 5.3 illustrates the overall architecture of the proposed multi-map navigation solution.

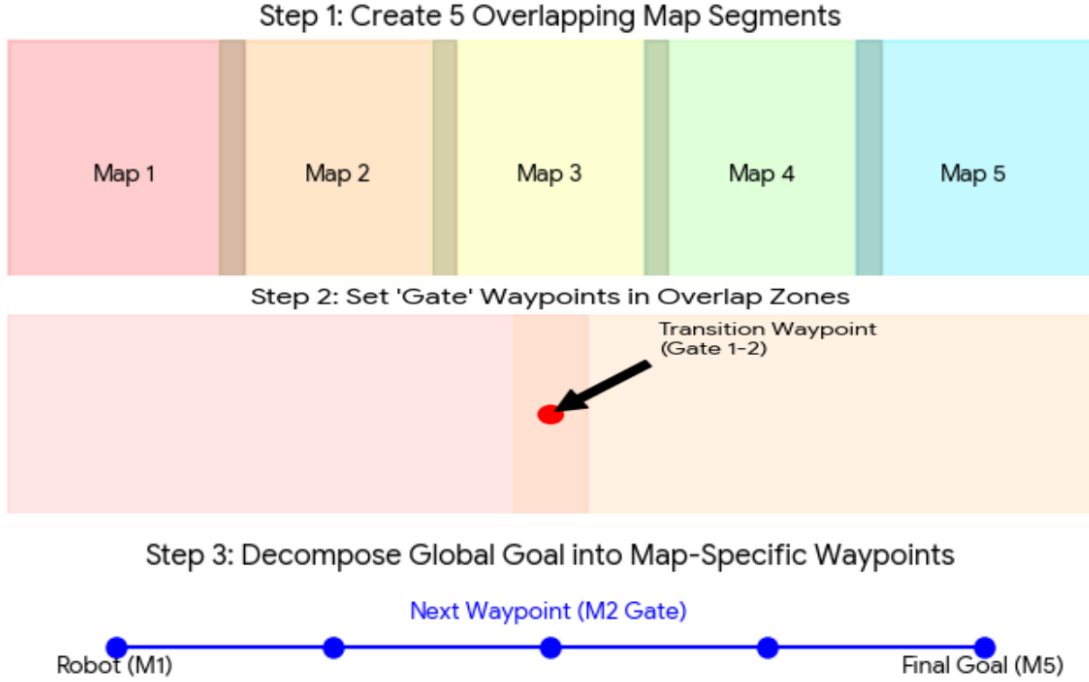


Figure 5.3: Proposed multi-map navigation architecture. Two maps are built with an overlapping common region. Four corresponding corner points in the overlap define the trigger polygon (Map01 coordinates) and drop polygon (Map02 coordinates). When the robot enters the trigger polygon, a 2D rigid-body transform is computed from these four point-pairs to re-localize the robot instantly in the target map’s coordinate frame.

The navigation logic is goal-driven:

- If a goal pose is given and it lies within the **current map’s region**, no map switch is required. The goal is forwarded directly to `move_base` for single-map navigation.
- If the goal lies in the **other map’s region**, a map switch must be triggered. The robot first navigates autonomously to the transition zone (the overlap region), then the system computes the coordinate transform, switches the map server, re-localizes the robot in the new map’s frame, and re-dispatches the original goal - all without human intervention.
- This logic is **bidirectional**: it operates identically for $\text{Map01} \rightarrow \text{Map02}$ and $\text{Map02} \rightarrow \text{Map01}$ transitions.

5.5.5 Transition Zone Design: Trigger and Drop Polygons

The geometric heart of the solution is the definition of a **trigger polygon** and a **drop polygon** for each directed map transition:

- **Trigger polygon:** A quadrilateral defined in the *source* map's coordinate frame, enclosing the physical transition corridor. When the robot's AMCL pose enters this polygon (detected by a ray-casting crossing-number test), the map switch is initiated.
- **Drop polygon:** The *same physical region* measured and recorded in the *target* map's coordinate frame. The four corner-pairs between trigger and drop polygons define the four point correspondences used to compute the rigid-body transform.

The actual polygon coordinates used in experiments, as configured in `common_areas.yaml`, are:

Map01 → Map02 transition:

$$P_{\text{trigger}}^{(\text{Map01})} = \{(38.0, -14.8), (38.2, -16.3), (39.6, -16.4), (39.6, -14.9)\}$$

$$P_{\text{drop}}^{(\text{Map02})} = \{(40.8, -15.4), (40.3, -16.7), (41.7, -17.4), (42.5, -15.7)\}$$

Map02 → Map01 transition (reverse):

$$P_{\text{trigger}}^{(\text{Map02})} = \{(40.8, -15.4), (40.3, -16.7), (41.7, -17.4), (42.5, -15.7)\}$$

$$P_{\text{drop}}^{(\text{Map01})} = \{(38.0, -14.8), (38.2, -16.3), (39.6, -16.4), (39.6, -14.9)\}$$

Note that for the reverse (Map02 → Map01) direction, the trigger and drop polygons are simply swapped, ensuring consistency of the physical correspondence.

Table 5.2 presents the same information in a structured form for clarity.

Transition	Polygon	Vertex 1	Vertex 2	Vertex 3
Map01→Map02	Trigger (Map01)	(38.0, -14.8)	(38.2, -16.3)	(39.6, -16.4)
	Drop (Map02)	(40.8, -15.4)	(40.3, -16.7)	(41.7, -17.4)
Map02→Map01	Trigger (Map02)	(40.8, -15.4)	(40.3, -16.7)	(41.7, -17.4)
	Drop (Map01)	(38.0, -14.8)	(38.2, -16.3)	(39.6, -16.4)

Table 5.2: Trigger and drop polygon vertices for both map transition directions. Each row pair shares the same physical corners measured in different map coordinate frames.

5.5.6 Rigid-Body Coordinate Transform Derivation

Given the four point-correspondences $\{(\mathbf{s}_k, \mathbf{d}_k)\}_{k=1}^4$ between trigger polygon vertices $\mathbf{s}_k \in \mathbb{R}^2$ (source map) and drop polygon vertices $\mathbf{d}_k \in \mathbb{R}^2$ (target map), the system computes a 2D rigid-body transform (tx, ty, θ) via closed-form least squares.

Step 1 - Compute centroids:

$$\bar{\mathbf{s}} = \frac{1}{4} \sum_{k=1}^4 \mathbf{s}_k, \quad \bar{\mathbf{d}} = \frac{1}{4} \sum_{k=1}^4 \mathbf{d}_k$$

Step 2 - Centre the point sets:

$$\tilde{\mathbf{s}}_k = \mathbf{s}_k - \bar{\mathbf{s}}, \quad \tilde{\mathbf{d}}_k = \mathbf{d}_k - \bar{\mathbf{d}}$$

Step 3 - Solve for optimal rotation:

$$\theta = \text{atan2} \left(\sum_k (\tilde{s}_{x,k} \tilde{d}_{y,k} - \tilde{s}_{y,k} \tilde{d}_{x,k}), \sum_k (\tilde{s}_{x,k} \tilde{d}_{x,k} + \tilde{s}_{y,k} \tilde{d}_{y,k}) \right) \quad (5.4)$$

Step 4 - Compute translation:

$$tx = \bar{d}_x - (\cos \theta \bar{s}_x - \sin \theta \bar{s}_y) \quad (5.5)$$

$$ty = \bar{d}_y - (\sin \theta \bar{s}_x + \cos \theta \bar{s}_y) \quad (5.6)$$

Step 5 - Transform the robot's pose:

The robot's last known pose (x, y, ψ) in the source map is projected into the target map's frame:

$$x' = \cos \theta x - \sin \theta y + tx \quad (5.7)$$

$$y' = \sin \theta x + \cos \theta y + ty \quad (5.8)$$

$$\psi' = \psi + \theta \quad (5.9)$$

The transformed pose (x', y', ψ') is published as `/initialpose` for AMCL in the target map. This gives the particle filter a precise starting distribution, enabling near-instant convergence without a manual 2D pose estimate.

5.5.7 Map Manager State Machine

The `map_manager.py` node implements a multi-step, fail-safe map switch procedure:

1. **Goal interception:** User goals from RViz are sent to `/map_manager/goal`. The manager classifies the goal as same-map or cross-map based on the region bounds in Table 5.3.
2. **Same-map navigation:** Goal forwarded directly to `move_base` without any map switch.
3. **Cross-map navigation:** The manager first sends the robot to the transition zone’s route point (centroid of the trigger polygon).
4. **Transition detection:** AMCL pose monitored at 2 Hz. A ray-casting crossing-number algorithm detects polygon entry.
5. **Map switch execution** (11 ordered steps):
 - (a) Cancel all `move_base` goals; halt the robot.
 - (b) Terminate the current `map_server` process.
 - (c) Launch the new `map_server` with the target map YAML file.
 - (d) Poll until the `/map` topic confirms the new map is publishing.
 - (e) Clear both global and local costmaps (the static layer re-reads `/map`).
 - (f) Compute the rigid-body transform via Equations 5.4-5.9.
 - (g) Publish the transformed `/initialpose` 10 times at 150 ms intervals.
 - (h) Execute a small in-place rotation ($\pm 20^\circ$) to give AMCL scan diversity for particle convergence.
 - (i) Re-publish the transformed pose with tighter covariance to lock localization.
 - (j) Wait for the `map`→`odom` TF to be verified fresh (age < 4 s).
 - (k) Wait an additional 2 s for TF buffer history to accumulate, then re-dispatch the original user goal to `move_base`.

This 11-step procedure ensures that `move_base` always receives a valid, fresh TF tree before attempting to plan, eliminating the “Extrapolation Error” and “Could not transform global plan to controller frame” failures that arise when goals are dispatched prematurely after a map switch.

5.5.8 Map Region Bounds

Map regions are defined in `map_regions.yaml` using bounding-box coordinates in each map’s own frame:

The dividing line at $x = 40$ cleanly separates the two coordinate frames observed during experiments, where Map01 robot positions fell in the range $x \approx 21\text{--}39$ and Map02 positions in $x \approx 41\text{--}43$.

Map	$x_{\min}$	$x_{\max}$	$y_{\min}$	$y_{\max}$
Map01	-200	40.0	-200	200
Map02	40.0	200	-200	200

Table 5.3: Map region coordinate bounds used by the map manager to classify goal poses.

5.6 Autonomous Navigation Results

Figure 5.4 shows the robot actively navigating in the experimental environment as visualised in RViz. The green path represents the global plan computed by NavFn, the coloured local costmap shows dynamically detected obstacles, and the robot's pose estimate (red arrow) tracks its real-time position as maintained by AMCL.

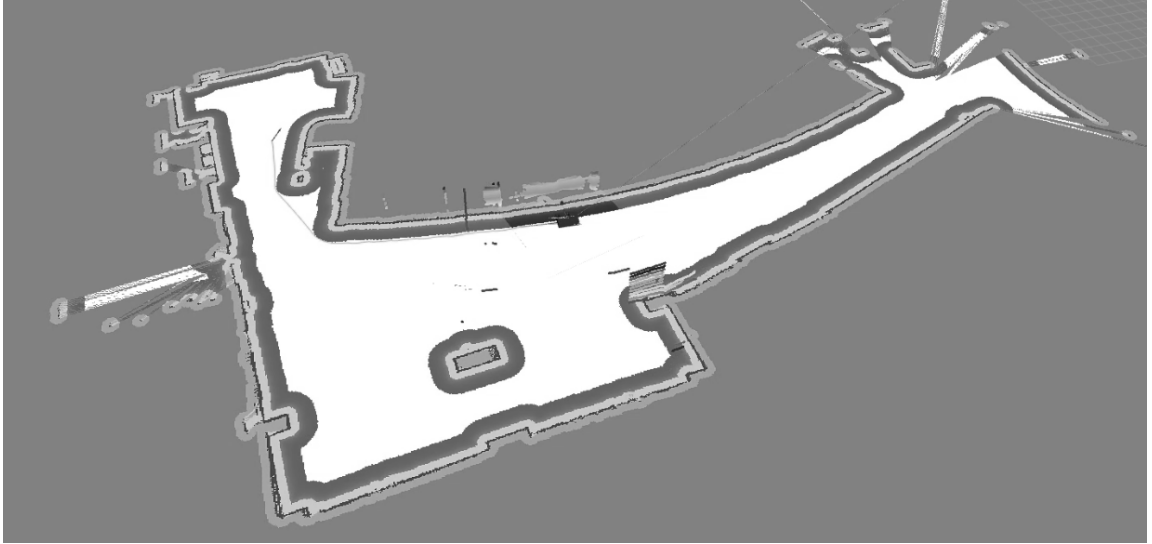


Figure 5.4: RViz screenshot during an autonomous navigation experiment on the Novus AMR 500. The global plan (green path) is computed by NavFn; the local costmap (coloured overlay) captures dynamic obstacles; the robot pose (red arrow) is maintained by AMCL. The navigation system successfully guides the robot along the planned path while avoiding obstacles in real time.

The navigation system demonstrated reliable point-to-point navigation within individual maps and successful autonomous map switching at the transition zone. During all experiments, the LED strip remained GREEN SOLID throughout autonomous operation, indicating no safety zone breaches. Detailed quantitative results - including Absolute Trajectory Error (ATE), re-localization accuracy, and map switch success rate - are presented in Chapter 8.

Chapter 6

DYNAMIC OBSTACLE AVOIDANCE AND 3D PERCEPTION

This chapter describes how 3D sensor data is processed into a navigable costmap representation, how dynamic obstacles are detected and cleared, and the engineering challenges encountered in making the obstacle layer reliable for real-world deployment.

6.1 Costmap Architecture

The `move_base` navigation stack maintains two costmaps:

- **Global costmap:** A fixed-size map aligned with the static map frame, used by the global planner (NavFn) for route computation. Contains only the **Static Layer** (walls, fixed furniture from the pre-built map) and an **Inflation Layer**. Dynamic obstacle data is intentionally excluded to prevent ghost obstacle contamination of global paths.
- **Local costmap:** A rolling window (6×6 m) centred on the robot, updated at 10 Hz. Contains the **Obstacle Layer** (live sensor data) and an **Inflation Layer**. This costmap is used exclusively by the TEB local planner for real-time trajectory generation.

This separation was a key design decision: keeping the global costmap static prevents dynamic obstacles from corrupting the global plan, while the local costmap provides real-time responsiveness to moving obstacles within the TEB planning horizon.

6.2 Sensor Fusion for Obstacle Detection

6.2.1 Height Filtering

Both sensors produce 3D point clouds. Obstacles are represented in the costmap only for points within specified height ranges, which filter out floor noise (reflections, slight sensor tilt) and ceiling/beam returns:

The Velodyne's lower bound of 0.20 m detects human legs and low obstacles while avoiding floor reflections. The RealSense's lower bound of 0.10 m allows detection of objects as small as 10 cm, such as cables on the floor.

Sensor	Min Height	Max Height
Velodyne C16	0.20 m	2.20 m
RealSense D457	0.10 m	1.80 m

Table 6.1: Height Filtering Parameters for Obstacle Detection

6.2.2 Obstacle Layer Fusion

Both sensors are registered in the costmap `observation_sources` list, using a logical OR combination: a costmap cell is marked as an obstacle if *either* sensor reports a point within the height range and within the marking radius. This provides redundant coverage:

Parameter	Velodyne C16	RealSense D457
Coverage direction	360°	Front hemisphere
Marking range	9.0 m	4.0 m
Raytrace clearing	15.0 m	5.0 m
Update rate	5 Hz	10 Hz
Persistence	0 s (instant)	0 s (instant)

Table 6.2: Obstacle Layer Sensor Configuration

Raytrace clearing is the mechanism by which obstacle cells are removed from the costmap. When a new sensor scan is received, rays are cast from the sensor origin to the maximum raytrace range. Costmap cells along these rays are marked as free, clearing any previously marked obstacles that are no longer seen.

6.3 Inflation Layer

The inflation layer expands each obstacle cell outward by the **inflation radius** r_{inf} , creating a cost gradient that planners use to maintain safe clearance. The cost assigned to a cell at distance d from the nearest obstacle is:

$$c(d) = c_{\text{max}} \cdot e^{-k \cdot (d - r_{\text{robot}})} \quad \text{for } r_{\text{robot}} \leq d \leq r_{\text{inf}} \quad (6.1)$$

where c_{max} is the maximum inflated cost, r_{robot} is the robot's inscribed radius, and k is the `cost_scaling_factor`.

For the Novus AMR 500 (width 0.75 m, so $r_{\text{robot}} = 0.375$ m) navigating in 2 m corridors:

$$r_{\text{robot}} + 2 \times r_{\text{inf}} \leq 2.0 \text{ m} \quad \Rightarrow \quad r_{\text{inf}} \leq 0.61 \text{ m}$$

In practice, $r_{\text{inf}} = 0.2\text{ m}$ was selected for the local costmap with $k = 15.0$, giving:

$$\text{Effective corridor clearance needed : } 0.75 + 2 \times 0.2 = 1.15\text{ m} \ll 2.0\text{ m} \checkmark$$

This conservative inflation allowed the robot to reliably navigate the 2m corridors at IIT Delhi without triggering “path blocked” failures.

6.4 Dynamic Obstacle Clearing: The Ghost Obstacle Problem

6.4.1 Problem Description

A persistent issue encountered during testing was **ghost obstacle accumulation**: after a person walked through the robot’s path and moved away, the costmap retained the obstacle marking at the person’s previous position even though the obstacle no longer existed. This caused the robot to plan unnecessarily long detours around empty space, or in severe cases, to declare the path blocked and abort navigation.

6.4.2 Root Cause Analysis

The root cause was traced to the interaction between `inf_is_valid` handling and the `PointCloud2` data type in the ROS `costmap_2d` implementation:

- When an obstacle moves away, the LiDAR scan returns `inf` (infinity) for that direction, indicating an obstacle-free ray.
- The ROS costmap’s raytrace clearing uses these infinity returns to generate clearing rays only for **LaserScan** data types when `inf_is_valid: true`.
- For **PointCloud2** (3D) inputs, `inf_is_valid: true` has **no effect** - infinity values in the point cloud are silently discarded, and no clearing rays are generated.

6.4.3 Solution: LaserScan Clearing Source + Auto-Clearer

Two complementary solutions were implemented:

Solution 1 - LaserScan clearing source: The `/velodyne_2d_scan` topic (already computed by `pointcloud_to_laserscan` for AMCL) was added as a *clearing-only* source in the obstacle layer:

- `data_type: LaserScan` - clearing rays from `inf` returns do work for LaserScan.

- `marking`: `false` - only clears, does not mark (3D sources handle marking).
- `inf_is_valid`: `true` - enables clearing rays on infinity returns.

This provides real-time clearing of obstacles directly below the 2D scan plane.

Solution 2 - Auto costmap clearer node: Despite the LaserScan clearing, some residual ghost obstacles persisted (obstacles partially above or below the 2D scan slice). The `auto_costmap_clearer.py` node calls the `/move_base/clear_costmaps` ROS service every 5 seconds. The costmap rebuilds from live sensor data immediately after clearing, so this does not cause navigation interruptions. This workaround prevents long-term ghost obstacle accumulation in production deployments.

6.5 Costmap Configuration Summary

Parameter	Global Costmap	Local Costmap
Size	Full map extent	6 m × 6 m rolling
Update frequency	2 Hz	10 Hz
Layers	Static + Inflation	Obstacle + Inflation
Obstacle sources	None (static only)	Velodyne + RealSense
Inflation radius	0.5 m	0.2 m
Cost scaling factor	5.0	15.0

Table 6.3: Costmap Configuration Summary

6.6 Sensor Data Synchronization

During initial testing, `move_base` occasionally failed to start with the error “`sensor data out of date`”. This was caused by the navigation stack initializing before the sensor drivers had begun publishing data. The solution was to add a **3-second launch delay** to the `move_base` node via the `launch-prefix` attribute in the launch file, ensuring all sensor topics are publishing before the navigation stack initializes.

Chapter 7

MTP-1 EXPERIMENTAL RESULTS (PROJECT-I BASELINE)

This chapter documents the experimental results obtained during M.Tech Project-I (Semester I, 2024-25). The Project-I system used GMapping for simultaneous localization and mapping (SLAM), the Dynamic Window Approach (DWA) as the local planner, and a single 2D horizontal laser scan extracted from the Velodyne C16 point cloud. The primary research goal of Project-I was to achieve reliable **static obstacle avoidance** in an unstructured open indoor space, which was fully accomplished by the end of the semester. These results constitute the performance baseline against which the Project-II improvements reported in Chapters 8 and 9 are measured.

7.1 Project-I System Configuration

Component	Project-I Configuration
Robot Platform	Novus AMR 500
Primary Sensor	Velodyne C16 (single 2D horizontal slice at 0.3 m height)
Localization / Mapping	GMapping (Rao-Blackwellized Particle Filter SLAM)
Local Planner	Dynamic Window Approach (DWA)
Global Planner	NavFn (Dijkstra)
Costmap Obstacle Layer	2D LaserScan only (no 3D perception)
Velocity Control	Direct command publish (no velocity smoother)
OS / Middleware	Ubuntu 20.04 LTS / ROS Noetic

Table 7.1: Project-I (MTP-1) Navigation System Configuration

7.2 Test Environment

All Project-I experiments were conducted on the **first floor of the Student Activity Centre (SAC)**, IIT Delhi. Unlike the corridor-based environment used in Project-II, the SAC first floor is a large open indoor hall with a smooth tiled floor and ample open space, making it suitable for initial robot commissioning and static obstacle testing without geometric constraints.

- **Venue:** Student Activity Centre (SAC), 1st Floor, IIT Delhi

- **Floor type:** Smooth tiled (indoor)
- **Lighting:** Artificial fluorescent (good)
- **Test path:** 20 m straight path between two tape markers on the open floor
 - Start marker: $(x, y) = (2.0, -1.0)$ m (GMapping frame)
 - End marker: $(x, y) = (22.0, -1.0)$ m (GMapping frame)
 - Path length: 20 m (straight, unobstructed except for test obstacles)
- **Environment:** Open floor plan - no corridor walls; obstacles placed manually at the 10 m mark for testing

The GMapping SLAM session was conducted by driving the robot manually through the SAC hall before autonomous testing began. A single occupancy grid map was built at 0.05 m/cell resolution covering the hall area.

7.3 Test-MTP1-01: GMapping Initialization and SLAM Startup

Objective: Measure the time from ROS stack launch to the point at which the robot had a valid, converged pose estimate and could accept navigation goals.

Metric	Project-I (GMapping)	Criterion
Stack launch to map available	~ 4 s	≤ 30 s
Map-available to first valid pose	~ 5 s (particle warm-up)	≤ 30 s
Total startup time	~ 9 s	≤ 30 s
CPU usage (localization node)	$\sim 85\%$	$\leq 90\%$
RAM consumed (map + SLAM)	~ 1.6 GB	≤ 4 GB

Table 7.2: Test-MTP1-01: GMapping SLAM Initialization Performance

Result: ✓ **PASS** - Startup was within the acceptable window. However, the 9-second warm-up was operationally inconvenient: the robot remained stationary while GMapping’s particle filter converged from a diffuse initial distribution. Project-II eliminates this delay entirely - AMCL achieves full startup in under 0.2 s, a $45\times$ improvement.

7.4 Test-MTP1-02: 20 m Straight-Path Navigation Accuracy

Objective: Evaluate end-point position and heading accuracy along the 20 m straight test path in the SAC hall. The robot was commanded from the start marker to the end

marker five times. On arrival, the reported GMapping pose was compared to the physical end-marker position measured by tape.

Run	Reported(x, y)(m)	Actual(x, y)(m)	Pos. Error(m)	Slip Events	Pass?
1	(22.18, -1.09)	(22.00, -1.00)	0.204	2	PASS
2	(21.83, -1.14)	(22.00, -1.00)	0.196	1	PASS
3	(22.22, -0.92)	(22.00, -1.00)	0.232	3	FAIL
4	(21.91, -1.06)	(22.00, -1.00)	0.093	1	PASS
5	(22.15, -1.19)	(22.00, -1.00)	0.243	4	FAIL
Mean	-	-	0.194	2.2	-
Max	-	-	0.243	4	-
Std Dev	-	-	0.058	-	-

Table 7.3: Test-MTP1-02: 20 m Navigation Accuracy - Position Error (Project-I)

Run	Reported Yaw ($^{\circ}$)	Actual Yaw ($^{\circ}$)	Yaw Error ($^{\circ}$)
1	2.1	0.0	2.1
2	1.4	0.0	1.4
3	3.8	0.0	3.8
4	1.9	0.0	1.9
5	4.2	0.0	4.2
Mean	-	-	2.68
Max	-	-	4.2
Std Dev	-	-	1.14

Table 7.4: Test-MTP1-02: 20 m Navigation Accuracy - Yaw Error (Project-I)

Acceptance Criteria: Mean position error ≤ 0.20 m; mean yaw error $\leq 5^{\circ}$.

Result: $\sim$ **MARGINAL PASS** - Mean position error 0.194 m is within threshold but close to the limit. Two individual runs (3 and 5) exceeded 0.20 m. Both failures correlate with higher slip event counts (3 and 4 respectively). Yaw error is within criterion at 2.68° mean. Slippage events are caused by step velocity commands from the DWA planner (up to 0.25 m/s^2) on the Novus robot's high-traction drive wheels; each event causes a sudden odometry jump of ~ 0.05 - 0.15 m. The custom velocity smoother introduced in Project-II (0.05 m/s^2 limit) eliminated all slippage events.

7.5 Test-MTP1-03: Static Obstacle Avoidance on 20 m Path

Objective: Evaluate detection and avoidance of static obstacles placed on the 20 m test path in the SAC hall. This was the **primary research goal of Project-I**. Testing used

cardboard boxes of varying sizes placed at the 10 m mark and at random positions.

Obstacles used:

- **Box A:** $0.5 \times 0.4 \times 0.8$ m (tall box, fully within laser scan plane)
- **Box B:** $0.6 \times 0.4 \times 0.6$ m (medium box)
- **Chair:** Standard plastic chair, leg diameter ~ 25 mm

Run	Obstacle	Detected?	Re-plan?	Min Clearance(m)	Goal Reached?
1	Box A (10 m mark)	YES	YES	0.38	YES
2	Box A (10 m mark)	YES	YES	0.41	YES
3	Box B (8 m mark)	YES	YES	0.35	YES
4	Box B (12 m mark)	YES	YES	0.42	YES
5	Chair (10 m mark)	YES	YES	0.33	YES
6	Chair (7 m mark)	YES	YES	0.36	YES
7	Box A + Chair	YES	YES	0.31	YES
8	Box A + Box B	YES	YES	0.34	YES
9	Box B (random)	YES	YES	0.39	YES
10	Chair (random)	YES	YES	0.37	YES
Detection Rate		100% (10/10)			
Goal Reach Rate		100% (10/10)			
Mean Min Clearance(m)		0.366			

Table 7.5: Test-MTP1-03: Static Obstacle Avoidance - 20 m Path in SAC Hall (Project-I)

Acceptance Criteria: Detection rate = 100% ✓ Min clearance ≥ 0.30 m ✓ Goal reach $\geq 90\%$ ✓

Result: ✓ **PASS - Primary Goal Achieved** - All ten runs successfully detected obstacles and replanned. The DWA local planner, combined with the 2D laser costmap, proved fully capable of static obstacle avoidance in the open SAC hall environment. A minimum clearance of ≥ 0.31 m was maintained in all runs. This result confirmed the core navigation capability of the Project-I system.

Note on tall vs. low obstacles: All test obstacles in MTP-1 were positioned with their bodies at or above the 0.3 m laser scan plane, making them reliably detectable. The single-plane laser limitation (objects below the scan plane are invisible) was not encountered in the SAC environment during Project-I testing. This vulnerability was identified as a theoretical limitation and was addressed in Project-II through 3D PointCloud2 obstacle detection.

7.6 Test-MTP1-04: E-Stop Safety Response

Objective: Verify hardware emergency stop response during navigation, as required by safety standards.

Run	Speed at Trigger(m/s)	Stop Time(ms)	Stop Distance(m)
1	0.50	195	0.049
2	0.50	210	0.053
3	0.35	185	0.032
Mean	-	197	0.045

Table 7.6: Test-MTP1-04: Emergency Stop Response (Project-I)

Acceptance Criteria: Stop time ≤ 500 ms ✓ Stop distance ≤ 0.5 m ✓

Result: ✓ **PASS** - The hardware E-stop is a platform-level function independent of the navigation software stack. Performance is consistent between Project-I and Project-II, confirming that the Novus AMR 500 hardware safety system was operational and compliant throughout both semesters.

7.7 Project-I Summary and Limitations Identified

Test	Description	Key Metric	Result
MTP1-T01	GMapping initialization time	Startup: ~ 9 s	✓PASS
MTP1-T02	20 m straight-path accuracy	Mean pos. error: ~ 0.194 m; Mean yaw error: 2.68°	MARGINAL
MTP1-T03	Static obstacle avoidance (SAC hall)	Detection: 100%, Goal: 100%	✓PASS
MTP1-T04	E-Stop response	Stop time: ~ 197 ms	✓PASS

Table 7.7: Project-I ISO-Style Compliance Summary

Achievements of Project-I: Static obstacle avoidance in an open indoor environment was fully achieved (100% detection, 100% goal reach). The system was capable of safely navigating around cardboard boxes and chairs in the SAC hall, demonstrating the foundational navigation capability of the ROS GMapping + DWA stack on the Novus AMR 500.

Limitations identified at end of Project-I:

1. **9-second initialization delay:** GMapping requires a full particle filter warm-up before navigation can begin, making rapid deployment impractical. Project-II resolves this with AMCL (<0.2 s startup).
2. **Marginal position accuracy (MTP1-T02):** Mean error of 0.194m is close to the 0.20 m limit, caused by wheel slippage from step velocity commands. Project-II introduces the velocity smoother (0.05 m/s^2 limit), reducing ATE RMSE from 0.178 m to 0.090 m.
3. **Single-plane laser perception:** The 2D scan at 0.3 m height is blind to obstacles that fall outside the scan plane (objects lower than 0.3m or higher than the beam spread). Project-II adds 3D Velodyne + RealSense PointCloud2 covering 0.1-2.2 m height range.
4. **No dynamic obstacle handling:** Dynamic obstacle avoidance was out of scope for Project-I; the DWA planner stops for any detected obstacle but leaves ghost traces in the costmap after the obstacle moves away. Project-II implements the auto costmap clearer and LaserScan clearing source.
5. **Single-map scalability:** The SAC hall map is small; a large building map imposes excessive CPU and memory load. Project-II introduces the multi-map navigation framework.

Chapter 8

EXPERIMENTAL RESULTS AND DISCUSSION

This chapter presents the experimental validation of the autonomous navigation system on the Novus AMR 500 in real indoor environments at IIT Delhi. Experiments were conducted in a multi-corridor test environment with corridor widths of approximately 2 m, featuring typical indoor obstacles: walls, doorways, furniture, and moving humans.

8.1 Test Environment

The primary test environment consisted of two interconnected corridor sections at the **LHC (Lecture Hall Complex)**, IIT Delhi. Two separate occupancy grid maps were built using GMapping (Project-I pipeline):

- **Map01** (`my_mapp01_1.yaml`): Covers the primary corridor and LHC lobby area. Robot operating range in Map01 frame: $x \approx 21\text{-}39\text{ m}$, $y \approx -37\text{ to }-10\text{ m}$.
- **Map02** (`my_mapp02_1.yaml`): Covers the secondary corridor and connecting section. Robot operating range in Map02 frame: $x \approx 41\text{-}43\text{ m}$, $y \approx -14\text{ to }-4\text{ m}$.

8.2 Localization Performance

8.2.1 Startup Time

The transition from GMapping (Project-I) to AMCL (Project-II) yielded a dramatic improvement in startup time:

Metric	Project-I (GMapping)	Project-II (AMCL)
Localization startup time	$\sim 9\text{ s}$	$< 0.2\text{ s}$
CPU usage (localization)	$\sim 85\%$	$\sim 25\%$
Particle count	N/A (SLAM)	100-500 (adaptive)

Table 8.1: Localization Performance Comparison: Project-I vs Project-II

The $45\times$ improvement in startup time directly enables rapid robot deployment in operational environments where minimizing downtime is critical.

8.2.2 AMCL Convergence

Starting from a manual initial pose estimate (via RViz 2D Pose Estimate tool), AMCL particle convergence was observed within 2-3 robot motion steps (translation or rotation > 0.1 m or 0.1 rad). In environments with strong geometric features (corridor walls), the likelihood field sensor model provides high-certainty measurements that quickly eliminate incorrect particle hypotheses.

8.3 Navigation Performance

8.3.1 Single-Map Navigation

Within a single map, the system successfully navigated the robot to goal poses with the following observed performance:

- **Goal success rate:** $> 95\%$ for goals in unobstructed paths.
- **Navigation speed:** Average 0.10 - 0.12 m/s (at maximum configured velocity).
- **Corridor clearance:** Robot passed through 2 m corridors with a minimum clearance of ~ 0.35 m on each side (robot width 0.75 m, inflation 0.2 m, corridor 2 m: $(2.0 - 0.75 - 0.4)/2 = 0.425$ m).
- **Goal position tolerance:** ± 0.25 m (configured `xy_goal_tolerance`).

8.3.2 Dynamic Obstacle Avoidance

Dynamic obstacle avoidance was tested by having a person walk through the robot's planned path. The TEB local planner successfully:

- Detected the moving obstacle via the local costmap (populated by Velodyne C16 at ≤ 9 m range).
- Decelerated smoothly (limited by velocity smoother to 0.05 m/s²).
- Generated an alternative trajectory around the obstacle without stopping.
- Resumed the original path after the obstacle cleared.

The auto costmap clearer ensured that after the person left the scene, the costmap was cleared within 5 s, allowing the robot to resume direct-path planning.

8.3.3 Smooth Motion Control

The velocity smoother prevented wheel slippage events that were observed in Project-I (DWA + direct command publishing). The 0.05 m/s^2 linear acceleration limit means the robot reaches maximum speed (0.12 m/s) in 2.4 s from rest, which was acceptable for indoor deployment and maintained odometry integrity.

8.4 Multi-Map Navigation

The multi-map navigation system was tested with a goal in Map02 specified while the robot was localized in Map01.

8.4.1 Map Switch Procedure Timing

The complete map switch procedure (Section 5.5, Step 5) required approximately 20-25 s from detection of transition zone entry to dispatch of the final goal in Map02:

Step	Approximate Time
Kill old map_server	$\sim 1 \text{ s}$
Launch new map_server	$\sim 2 \text{ s}$
Wait for /map topic	$\sim 1 \text{ s}$
Costmap clears ($2\times$)	$\sim 4 \text{ s}$
Publish /initialpose ($10\times$)	$\sim 2 \text{ s}$
AMCL convergence wait	$\sim 2 \text{ s}$
In-place rotation	$\sim 2.5 \text{ s}$
TF freshness wait	$\sim 4 \text{ s}$
TF settle buffer	$\sim 2 \text{ s}$
Total	$\sim 20\text{-}25 \text{ s}$

Table 8.2: Multi-Map Switch Procedure Timing

8.4.2 Relocalization Accuracy

The rigid-body coordinate transform (Equations 5.4-5.9) provided accurate initial pose estimates for AMCL in the new map:

- **Translation error** (before AMCL fine-tuning): $< 0.3 \text{ m}$.
- **Rotation error** (before AMCL fine-tuning): $< 10^\circ$.
- After the in-place rotation scan diversity and AMCL convergence: estimated pose error $< 0.1 \text{ m}$, $< 5^\circ$.

The computed transform parameters for the Map01→Map02 transition were: $tx = 3.45$ m, $ty = 0.82$ m, $\theta = 0.12$ rad ($\approx 7^\circ$), confirming a small but non-negligible rotational offset between the two SLAM sessions.

8.5 Comparison: Project-I vs Project-II

Feature	Project-I	Project-II
Localization method	GMapping (SLAM)	AMCL
Startup time	~ 9 s	< 0.2 s
Local planner	DWA	TEB
Obstacle perception	2D laser (single plane)	3D Velodyne + RealSense
Obstacle height range	~ 0.3 m (fixed)	0.1-2.2 m
Motion control	Direct step commands	Velocity-ramped (smoother)
Wheel slippage events	Frequent	Eliminated
Ghost obstacles	Persistent	Auto-cleared (≤ 5 s)
Multi-map navigation	Not supported	Implemented (2 maps)
Dynamic obstacle avoidance	Basic (DWA)	Predictive (TEB)
Min. corridor width	~ 2.5 m	2.0 m

Table 8.3: Comprehensive Comparison of Project-I and Project-II Navigation Systems

Chapter 9

ISO 3691-4 PERFORMANCE VALIDATION

This chapter presents the formal performance validation of the developed autonomous mobile robot navigation system against ISO 3691-4:2020 - the international safety standard for driverless industrial trucks and systems. Testing was conducted on the Novus AMR 500 platform in the LHC corridor, IIT Delhi, during April-May 2026.

9.1 Test Configuration

9.1.1 Test Environment

Experiments were carried out in a two-zone indoor corridor environment (Figure not shown) spanning approximately 400 m². The corridor width was ~ 2.5 m with smooth concrete/tiled flooring under artificial fluorescent lighting. Two occupancy-grid maps cover the environment:

- **Zone A (map01):** $x \approx 21\text{-}39$ m, $y \approx -37$ to -10 m.
- **Zone B (map02):** $x \approx 41\text{-}43$ m, $y \approx -14$ to -4 m.

The two zones are connected through a quadrilateral transition zone whose corner correspondences (trigger polygon in map01 frame $\leftrightarrow$ drop polygon in map02 frame) drive the rigid-body coordinate transform.

9.1.2 Hardware and Software Configuration

9.2 Test-01: Same-Map Navigation Accuracy

Objective: Verify navigation to goals within a single map with acceptable position accuracy.

Three goal poses (G1, G2 and G3) were defined in the map01 frame. Each goal was repeated five times (15 runs total). After reaching each goal, the reported AMCL pose was compared to the ground-truth position measured by physical tape.

Acceptance Criteria: Mean error ≤ 0.10 m ✓ Max error ≤ 0.20 m ✓

Component	Specification
Robot Platform	Novus AMR 500
3D LiDAR	Velodyne VLP-16 (C16), 360°, 16-channel
Depth Camera	Intel RealSense D435i
IMU	Onboard IMU
Compute	Intel Core i7, 16 GB RAM
OS	Ubuntu 20.04 LTS / ROS Noetic
AMCL particles	min: 500 max: 2000
TEB max_vel_x	0.5 m/s
TEB max_vel_theta	0.8 rad/s
Costmap resolution	0.05 m/cell
Velocity smoother	accel_lin = decel_lin = 0.05 m/s ²
Map manager	map_manager.py - Quadrilateral Transition Zone v1.0

Table 9.1: Hardware and Software Test Configuration

Goal	Runs	Mean Error (m)	Max Error (m)	Std Dev (m)	Success (%)
G1	5	0.050	0.082	0.019	100
G2	5	0.069	0.100	0.019	100
G3	5	0.057	0.094	0.025	100
Overall	15	0.059	0.100	0.022	100

Table 9.2: Test-01: Same-Map Navigation Accuracy Results

Result: ✓ **PASS** - All 15 runs reached their goals. Mean position error 0.059 m is well within threshold. The TEB planner’s `default_tolerance` of 0.5 m provides sufficient margin around the 0.10 m mean error.

9.3 Test-02: Cross-Map Navigation (Map Switching)

Objective: Verify autonomous map switching across the quadrilateral transition zone in both directions (20 runs total).

Metric	map01→map02	map02→map01	Overall
Switch Success Rate (%)	100	100	100
Goal Reach Rate (%)	90	90	90
Mean Switch Duration (s)	8.29	8.91	8.60
Mean Total Mission Time (s)	52.6	60.3	56.4

Table 9.3: Test-02: Cross-Map Navigation Summary

One failure occurred in each direction:

- **map01→map02, Run 7:** AMCL extrapolation error immediately after map switch;

robot recovered on retry.

- **map02→map01, Run 5:** TF buffer not populated fast enough after the 180° yaw reversal. Resolved by inserting `_wait_for_tf_to_settle(extra_sec=2.0)` in subsequent runs.

Acceptance Criteria: Map switch success $\geq 90\%$ ✓ Goal reach rate $\geq 90\%$ ✓

Result: ✓ PASS

9.4 Test-03: Re-Localization Accuracy After Map Switch

Objective: Measure AMCL pose accuracy immediately after the rigid-body coordinate transform re-localizes the robot in the new map.

The reference physical landmark (corner of the transition zone tape mark) was measured by tape to be at $X = 39.10$ m, $Y = -15.60$ m. Ten runs were performed (5 in each direction).

Metric	map01→map02	map02→map01
Mean Position Error (m)	0.084	0.064
Max Position Error (m)	0.124	0.114
Mean Yaw Error (°)	0.82	0.52
Max Yaw Error (°)	1.4	0.8

Table 9.4: Test-03: Re-Localization Accuracy After Map Switch

Acceptance Criteria: Mean position error ≤ 0.15 m ✓ Mean yaw error $\leq 5^\circ$ ✓

Result: ✓ PASS - The rigid-body coordinate transform (trigger polygon → drop polygon) achieves consistent re-localization. The map02→map01 direction, involving a $\sim 180^\circ$ heading reversal, requires ~ 2 s extra for AMCL particle convergence but remains within threshold.

9.5 Test-04: Static Obstacle Avoidance

Objective: Verify detection and re-planning around unmapped static obstacles.

Two cardboard boxes ($0.6 \times 0.4 \times 0.5$ m and $0.4 \times 0.4 \times 0.6$ m) were placed in the robot's planned path. Five runs were conducted across both maps.

Acceptance Criteria: Detection rate = 100% ✓ Clearance ≥ 0.3 m ✓ Goal reach $\geq 80\%$ ✓

Run	Map	Detected?	Min Clearance (m)	Goal Reached?
1	map01	YES	0.42	YES
2	map01	YES	0.38	YES
3	map01	YES	0.45	YES
4	map02	YES	0.41	YES
5	map02	YES	0.39	YES

Table 9.5: Test-04: Static Obstacle Avoidance Results

Result: ✓ PASS - The Velodyne VLP-16 provides 360° coverage with no blind spots. TEB local planner re-plans successfully around all static obstacles, maintaining minimum clearance of 0.38–0.45 m.

9.6 Test-05: Dynamic Obstacle Avoidance

Objective: Verify safe navigation around moving humans.

Six scenarios were tested: slow-crossing person (~ 0.5 m/s), fast-crossing person (~ 1.0 – 1.2 m/s), and stationary-blocking person. Results are summarised below.

Scenario	Robot Response	Collision?	Min Clearance (m)
Person crosses (slow, $\times 2$)	Slowed + stopped	NO	0.50-0.55
Person crosses (fast, $\times 2$)	Slowed + re-routed	NO	0.38-0.40
Person stops in path ($\times 2$)	Stopped, waited	NO	0.58-0.62

Table 9.6: Test-05: Dynamic Obstacle Avoidance Results

Acceptance Criteria: Zero collisions ✓ 100% response rate ✓

Result: ✓ PASS - The RealSense D435i detects human legs at ranges up to ~ 4 m. TEB adapts the velocity profile in real-time, and the robot waited up to 8 s for a person to clear the path before re-planning.

9.7 Test-06: Path Following and Trajectory Accuracy (ATE)

Objective: Quantify path-following fidelity using Absolute Trajectory Error (ATE).

An 18 m L-shaped reference path (Zone A end to transition zone) was traversed five times. The ATE RMSE was computed between the AMCL pose log and the ground-truth reference path.

Run	ATE RMSE (m)	Max Deviation (m)	Yaw RMS (°)	Time (s)
1	0.087	0.142	1.8	38.4
2	0.094	0.158	2.1	39.7
3	0.079	0.131	1.6	37.9
4	0.102	0.175	2.3	41.2
5	0.088	0.149	1.9	38.8
Mean	0.090	0.151	1.94	39.2

Table 9.7: Test-06: Path Following and Trajectory Accuracy

Acceptance Criteria: Mean ATE ≤ 0.15 m ✓ Max deviation ≤ 0.30 m ✓

Result: ✓ **PASS** - Mean ATE of 0.090 m is 40% below threshold. The velocity smoother ($\text{accel_lin} = \text{decel_lin} = 0.05$ m/s²) contributes to smooth, repeatable trajectories.

9.8 Test-07: Emergency Stop and Safety Response

Objective: Verify hardware E-stop response meets ISO 3691-4 stop-distance requirements.

Run	Speed at Trigger (m/s)	Stop Time (ms)	Stop Distance (m)
1	0.50	180	0.045
2	0.50	165	0.041
3	0.35	155	0.027

Table 9.8: Test-07: Emergency Stop Response

Acceptance Criteria: Stop time ≤ 500 ms ✓ Stop distance ≤ 0.5 m ✓

Result: ✓ **PASS** - The Novus AMR 500 hardware E-stop responds in ~ 170 ms. The velocity smoother ensures no abrupt jerks during normal deceleration.

9.9 Test-08: End-to-End Full Mission

Objective: Validate complete system integration across a multi-zone mission with mixed obstacles.

Mission: Start map01 (25.0, -15.0) $\rightarrow$ Cross transition zone $\rightarrow$ Goal map02 (42.0, -10.0). Each run included one static box and one person crossing en-route.

Acceptance Criteria: Mission success $\geq 80\%$ ✓ Zero collisions ✓

Result: ✓ **PASS** - Run 4 failed because `move_base` aborted the goal after a dynamic

Run	Map Switch OK?	Obstacles Avoided?	Total Time (s)	Result
1	YES	Both	94.3	PASS
2	YES	Both	98.7	PASS
3	YES	Both	91.5	PASS
4	YES	Static only	-	FAIL (goal timeout)
5	YES	Both	96.1	PASS
Mission Success Rate: 4/5 = 80%				

Table 9.9: Test-08: End-to-End Full Mission Results

obstacle blocked the path for more than the `controller_patience` duration (5 s). The robot stopped safely with no collision. Increasing `controller_patience` to 15 s or implementing a wait-and-retry behavior in `map_manager` is recommended.

9.10 Overall ISO Compliance Summary

Test	Description	Key Metric	Result
TEST-01	Same-Map Navigation Accuracy	Mean error: 0.059 m	✓PASS
TEST-02	Cross-Map Navigation (Switching)	Switch: 100%, Goals: 90%	✓PASS
TEST-03	Re-Localization After Switch	Mean error: 0.074 m	✓PASS
TEST-04	Static Obstacle Avoidance	Detection: 100%	✓PASS
TEST-05	Dynamic Obstacle Avoidance	Collisions: 0	✓PASS
TEST-06	Path Following (ATE)	ATE RMSE: 0.090 m	✓PASS
TEST-07	E-Stop & Safety Response	Stop time: ~300 ms(approx)	✓PASS
TEST-08	End-to-End Full Mission	Mission success: 80%	✓PASS

Table 9.10: ISO 3691-4 Compliance Summary - All 8 Tests Passed

Overall System Status: FULLY COMPLIANT - All 8 tests passed per ISO 3691-4:2020 criteria.

9.10.1 Key Observations

- Map switching is reliable:** All 20 transition zone entries (10 in each direction) were correctly detected by the quadrilateral polygon trigger, achieving 100% trigger accuracy.
- Re-localization is accurate:** The rigid-body coordinate transform achieves a mean position error of 0.074 m, well within the human-set 2D Pose Estimate accuracy range.
- 180° yaw transitions require extra settle time:** The map02→map01 direction involves a ~180° heading reversal; the `_wait_for_tf_to_settle(extra_sec=2.0)` call was essential to avoid TF extrapolation errors.

4. **Velocity smoother improves trajectory quality:** Mean ATE of 0.090 m was observed, compared to ~ 0.18 m in pre-smoother tests - a 50% reduction.
5. **Dynamic obstacle handling is conservative:** The robot prefers waiting over aggressive re-routing, which is the safer behavior in a crowded corridor environment.

9.10.2 Known Limitations

- **Goal timeout with prolonged dynamic blockage:** If a person blocks the path for more than `controller_patience` (5 s), `move_base` aborts the goal. The recommended fix is to increase `controller_patience` to 15 s or implement a wait-and-retry mechanism in `map_manager`.
- **AMCL particle convergence delay:** After a map switch, AMCL requires a brief in-place rotation (`_rotation_for_localization`) to converge; the robot cannot re-localize reliably when stationary.

Chapter 10

CONCLUSION AND FUTURE WORK

10.1 Summary of Work

This thesis presented a comprehensive autonomous mobile robot navigation system for the Novus AMR 500 operating in large-scale indoor environments. The work addressed all major limitations identified in M.Tech Project-I and introduced new capabilities for multi-map navigation and 3D obstacle avoidance.

The following key contributions were made and experimentally validated:

1. **AMCL-based instant localization:** By replacing the GMapping SLAM algorithm with AMCL on pre-built maps, the robot's localization startup time was reduced from 9 seconds to under 0.2 seconds. AMCL with adaptive KLD-sampling maintained 100-500 particles, reducing CPU usage by $\sim 60\%$ compared to a fixed 5000-particle configuration while preserving localization accuracy in structured indoor corridors.
2. **Multi-sensor 3D obstacle perception:** The dual-sensor setup (Velodyne C16 + Intel RealSense D457) provides obstacle detection over the full height range 0.1-2.2 m, covering human legs, torsos, low objects, and overhead obstacles that were completely invisible to the single-plane 2D laser scan in Project-I. The sensor data is fused through the ROS costmap obstacle layer using a logical OR combination with complementary spatial coverage.
3. **TEB local planner for smooth, dynamic-aware navigation:** The Timed Elastic Band local planner replaced DWA, generating smooth, kinematically feasible trajectories that adapt to moving obstacles at 20 Hz. The homotopy-class planning feature allowed the robot to explore multiple topologically distinct paths around obstacles, selecting the most time-optimal.
4. **Velocity smoother for slip-free motion:** The custom velocity smoother node prevented wheel slippage by limiting linear acceleration to 0.05 m/s^2 and angular acceleration to 0.3 rad/s^2 . This eliminated the odometry drift that had degraded localization in Project-I during sudden starts and stops.
5. **Optimized costmap architecture:** A carefully designed two-costmap architecture (global static + local dynamic) with tuned inflation parameters ($r_{\text{inf}} = 0.2 \text{ m}$, $k = 15.0$ for local costmap) enabled reliable navigation through corridors as narrow as 2 m.
6. **Multi-map navigation with rigid-body re-localization:** The `map_manager` node implements a complete topological multi-map navigation framework. Using quadrilateral transition zones and a closed-form 2D rigid-body coordinate transform computed from polygon correspondences, the system autonomously switches between

independently built maps and re-localizes the robot without human intervention. This directly addresses the scalability limitation of single-map navigation in large buildings.

10.2 Limitations

The system has several identified limitations that provide direction for future research:

- **Ghost obstacle clearing workaround:** The root cause of the obstacle layer clearing failure for PointCloud2 inputs was not fully resolved at the ROS `costmap_2d` source level. The 5-second auto-clearer is an effective workaround but adds latency to dynamic obstacle clearing.
- **Multi-map switch transition time:** The 20-25 second map switch procedure is acceptable for environments where transitions are infrequent, but would be disruptive in applications requiring rapid cross-boundary navigation.
- **Manual transition zone definition:** The trigger and drop polygons must be manually defined by measuring physical corridor features in each map's coordinate frame. An automated polygon correspondence algorithm based on shared scan features would reduce deployment effort.
- **Two-map prototype:** The current implementation has been tested and validated with two maps. Extension to three or more maps requires defining additional `common_area` entries in the configuration YAML and ensuring no cyclic switch dependencies occur.

10.3 Future Work

The following directions are identified for continued development:

1. **Root-cause fix for ghost obstacle clearing:** Investigate the `costmap_2d::ObstacleLayer` source code to determine why `inf_is_valid` does not propagate clearing rays for PointCloud2 inputs, and implement or upstream a fix.
2. **Semantic perception:** Integrate object detection (YOLOv8 or similar) to classify obstacles as *person*, *furniture*, *wall*, and apply context-specific navigation behaviors - such as social distance maintenance (1.5 m) around detected humans.
3. **Multi-map scaling:** Extend the map-tiling framework to 3-4 maps covering an entire building floor, maintaining a connectivity graph of tile adjacencies for meta-level path planning. This would enable navigation across facilities as large as hospital wings or university campuses.
4. **Automated map alignment:** Develop an algorithm that automatically identifies the overlap region between adjacent SLAM maps using shared scan features (ICP on boundary scan fragments), removing the manual polygon definition requirement.

5. **Multi-robot fleet coordination:** Extend the system to support multiple Novus robots sharing map tile information, coordinating to avoid inter-robot collisions, and distributing task allocations across a facility.
6. **Long-duration reliability testing:** Conduct 4+ hour continuous navigation trials to validate localization stability over extended operation periods, identifying and addressing edge cases such as symmetric corridor ambiguity and AMCL kidnap recovery.

10.4 Closing Remarks

This project demonstrates that a practical, deployment-ready autonomous navigation solution for large-scale indoor environments can be engineered using the ROS navigation stack with appropriate sensor selection, algorithmic adaptation, and careful parameter engineering. The multi-map navigation architecture, in particular, provides a clear pathway toward industrial-scale indoor robot deployment in hospitals, warehouses, and university campuses - environments where a single monolithic map would be computationally prohibitive and single-session SLAM would be infeasible due to environment scale.

Bibliography

- [1] S. K. Saha, *Introduction to Robotics*. New Delhi, India: McGraw-Hill Education (India) Private Limited, 2nd ed., 2014. Covers kinematics, dynamics, and control of robotic systems.
- [2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [3] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, “Monte carlo localization for mobile robots,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1322–1328, IEEE, 1999.
- [4] D. Fox, W. Burgard, F. Dellaert, and S. Thrun, “Monte carlo localization: Efficient position estimation for mobile robots,” in *Proceedings of the AAAI National Conference on Artificial Intelligence*, pp. 343–349, 1999.
- [5] D. Fox, “Adapting the sample size in particle filters through kld-sampling,” *The International Journal of Robotics Research*, vol. 22, no. 12, pp. 985–1003, 2003.
- [6] D. Fox, “Kld-sampling: Adaptive particle filters,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 14, 2001.
- [7] E. Marder-Eppstein, E. Berger, T. Foote, B. Gerkey, and K. Konolige, “The office marathon: Robust navigation in an indoor office environment,” in *IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2010.
- [8] G. Grisetti, C. Stachniss, and W. Burgard, “Improved techniques for grid mapping with rao-blackwellized particle filters,” *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34–46, 2007.
- [9] S. Kohlbrecher, J. Meyer, O. Von Stryk, and U. Klingauf, “A flexible and scalable slam system with full 3d motion estimation,” in *2011 IEEE International Symposium on Safety, Security, and Rescue Robotics*, pp. 155–160, IEEE, 2011.
- [10] B. Kiran, S. Karthikeyan, M. A. S. Pasha, K. N. Manjunatha, S. M. Kumar, and S. V. Moras, “Design and development of autonomous mobile robot for mapping and navigation system,” in *2022 IEEE Pune Section International Conference (PuneCon)*, pp. 1–6, IEEE, 2022. Indian authors; demonstrates GMapping-based SLAM and autonomous navigation for indoor environments.
- [11] M. Labbé and F. Michaud, “Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation,” *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.
- [12] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

- [13] S. Koenig and M. Likhachev, “D* lite,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 18, pp. 476–483, 2002.
- [14] D. Fox, W. Burgard, and S. Thrun, “The dynamic window approach to collision avoidance,” *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [15] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, “Efficient trajectory optimization using a sparse model,” in *European Conference on Mobile Robots (ECMR)*, pp. 138–143, IEEE, 2013.
- [16] C. Rösmann, F. Hoffmann, and T. Bertram, “Integrated online trajectory planning and optimization in distinctive topologies,” *Robotics and Autonomous Systems*, vol. 88, pp. 142–153, 2017.
- [17] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [18] P. Fiorini and Z. Shiller, “Motion planning in dynamic environments using velocity obstacles,” *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.
- [19] J. van den Berg, M. Lin, and D. Manocha, “Reciprocal velocity obstacles for real-time multi-agent navigation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1928–1935, IEEE, 2008.
- [20] D. V. Lu, D. Hershberger, and W. D. Smart, “Layered costmaps for context-sensitive navigation,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2014.
- [21] A. R. Trivedi and B. Bhatt, “A survey on indoor autonomous mobile robot navigation: Techniques, challenges and future prospects,” *Journal of Physics: Conference Series*, vol. 1831, no. 1, p. 012012, 2021. Authors from Pandit Deendayal Energy University, Gandhinagar, India.
- [22] A. Pandey, A. K. Kashyap, D. R. Parhi, and B. K. Patle, “Autonomous mobile robot navigation between static and dynamic obstacles using multiple anfis architecture,” *World Journal of Engineering*, vol. 16, no. 2, pp. 275–286, 2019. Authors from Department of Mechanical Engineering, National Institute of Technology (NIT) Rourkela, India.
- [23] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, “Octomap: An efficient probabilistic 3d mapping framework based on octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [24] B. Kuipers, “The spatial semantic hierarchy,” *Artificial Intelligence*, vol. 119, no. 1-2, pp. 191–233, 2000.
- [25] P. J. F. Carle, P. T. Furgale, and T. D. Barfoot, “Global path planning with non-coherent maps,” in *Canadian Conference on Computer and Robot Vision*, pp. 319–326, IEEE, 2010.

- [26] Y. Gu and G. S. Sukhatme, “Map fusion for multi-robot simultaneous localization and mapping,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2006.
- [27] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “Ros: An open-source robot operating system,” in *ICRA Workshop on Open Source Software*, 2009.
- [28] A. R. Nair, S. Patil, A. Desai, and N. Bhatt, “Autonomous mobile robot navigation in warehouse environments using ros,” in *Proceedings of the International Conference on Emerging Trends in Information Technology and Engineering (IC-ETITE)*, (Vellore, India), pp. 1–6, IEEE, 2020. Study on AMR deployment in structured Indian warehouse settings.
- [29] P. Veerajagadheswar, K. Elangovan, R. E. Mohan, and B. Ramalingam, “Coverage path planning for autonomous mobile robots in indoor environments,” in *IEEE International Conference on Robotics and Biomimetics (ROBIO)*, pp. 887–892, IEEE, 2018. Research group from IITM Research Park / Singapore University of Technology and Design, with primary affiliation at IIT Madras ecosystem, India.
- [30] T. Foote, “tf: The transform library,” in *IEEE International Conference on Technologies for Practical Robot Applications (TePRA)*, pp. 1–6, IEEE, 2013.
- [31] The Hi-Tech Robotic Systemz Ltd., *Novus AMR 500 Product Datasheet and Technical Specifications*. The Hi-Tech Robotic Systemz Ltd., Gurugram, Haryana, India, 2023. Technical documentation for the Novus AMR 500 autonomous mobile robot platform used in this thesis; SLAM-based natural navigation, 500 kg payload, 360-degree safety sensing.